Índice general

Resumen I

1. Introducción general 1
1.1. Extracción de información musical . . . 1
1.2. Contexto de aplicación . . . 1
1.3. Objetivos y alcance . . . 2
1.3.1. Objetivo general . . . 2
1.3.2. Objetivos específicos . . . 2
1.3.3. Alcance . . . 2
1.4. Estado del arte . . . 3
1.4.1. Procesamiento digital de señales y extracción a bajo nivel . . . 3
1.4.2. Modelos preentrenados y soluciones comerciales . . . 3
1.4.3. Síntesis comparativa . . . 4

2. Introducción específica 5
2.1. Fuentes de datos de canciones . . . 5
2.2. Representaciones numéricas de la música . . . 5
2.3. Información semántica del audio . . . 7
2.4. Modelos de inteligencia artificial utilizados . . . 8
2.5. Despliegue e integración . . . 9

3. Diseño e implementación 11
3.1. Conjuntos de datos y extracción de atributos . . . 11
3.1.1. Análisis comparativo de conjuntos de datos públicos . . . 11
GTZAN . . . 11
FMA estándar . . . 12
Million Song Dataset (MSD) . . . 13
AudioSet . . . 13
3.1.2. Reducción de dimensionalidad mediante estadísticas . . . 13
3.1.3. Definición del espacio de variables . . . 16
3.2. Ejecución de tareas sobre el conjunto de datos . . . 17
3.2.1. Distribución y tratamiento de las variables objetivo . . . 17
Loudness . . . 17
Key y mode . . . 18
Tempo . . . 19
Acousticness, danceability, energy, instrumentalness y valence . . . 20
Speechiness . . . 21
Liveness . . . 22
Top-5 primary mood . . . 23
Top-5 secondary mood . . . 24
Suggested genres . . . 25

3.2.2. Selección de variables de entrada 26
3.3. Modelado y optimización 27
Modelado mediante perceptrón multicapa (MLP) 27
Modelado mediante Gradient Boosting (LightGBM) 28
Optimización de hiperparámetros y regularización 29
3.4. Arquitectura de despliegue e integración 30
3.4.1. Diseño del servicio predictivo 30
3.4.2. Flujo de integración 31

4. Ensayos y resultados **33**
4.1. Resultados por modelo 33
4.1.1. Acousticness, danceability, energy, instrumentalness y valence 33
4.1.2. Loudness 35
4.1.3. Key y mode 37
4.1.4. Tempo 38
4.1.5. Speechiness y liveness 40
4.1.6. Suggested Genres 42
4.1.7. Primary y secondary moods 44
4.2. Ejemplo de inferencia 46

5. Conclusiones **49**
5.1. Conclusiones generales 49
5.2. Próximos pasos 50

Bibliografía **51**

Índice de figuras

1.1. Interfaz de consulta en Tunebat 4

2.1. Representaciones visuales del audio 6
2.2. Probabilidades de detección semántica 8

3.1. Algoritmo de extracción de atributos 14
3.2. Proceso de reducción de dimensionalidad 15
3.3. Distribución de loudness 18
3.4. Distribución de key y mode 18
3.5. Distribución de tempo 19
3.6. Distribuciones de características perceptibles 20
3.7. Distribución de speechiness 21
3.8. Distribución de liveness 22
3.9. Distribución de primary mood 23
3.10. Distribución de freciencias secondary mood 24
3.11. Distribución de frecuencias de suggested genres 25
3.12. Bloques del perceptrón multicapa para las variables perceptibles. . . 27
3.13. Resumen del modelado LightGBM. 28
3.14. Flujo de optimización de parámetros para los modelos LightGBM. . 29
3.15. Diagrama de bloques de la aplicación predictiva. 30
3.16. Diagrama de despliegue e integración. 31
4.1. Curvas de aprendizaje del modelo MLP. 31
4.2. Relación de valores reales y estimados en el modelo MLP. 32
4.3. Top-15 variables predictivas del modelo MLP. 32
4.4. Curvas de aprendizaje para loudness. 33
4.5. Curvas de aprendizaje del modelo MLP para Loudness. 33
4.6. Relación de valores reales y estimados en el modelo MLP. 34
4.7. Top-15 variables predictivas del modelo MLP. 35
4.8. Curvas de aprendizaje para key y mode. 35
4.9. Relación de valores reales y estimados para Loudness. 36
4.10. Top-15 variables predictivas para loudness. 36
4.11. Matriz de confusión de key y mode. 37
4.12. Relación de valores actuales y estimados para tempo. 39
4.13. Relaciones lineales entre el tempo actual y el estimado con Essentia. 39
4.14. Histogramas de frecuencias del tempo actual y el estimado con Essentia. 39
4.15. Top-15 variables predictivas para liveness. 39
4.16. Pruebas preliminares (izquierda) y actuales (derecha) para speechiness y liveness. 40
4.17. Matriz de confusión normalizada para la clasificación de 197 géneros musicales. 41
4.18. Top-15 variables predictivas para speechiness. 41
4.19. Top-15 variables predictivas para liveness. 42
4.20. Curvas de aprendizaje del modelo para la clasificación de géneros. 42
4.21. Matriz de confusión normalizada para la clasificación de 197 géneros musicales. 43
4.22. Top-15 variables predictivas para género. 43
4.23. Matriz de confusión para primary mood. 44

4.19. Curvas ROC y valores AUC para primary mood. 44
4.20. Matriz de confusión normalizada de secondary mood. 45
4.21. Visualización de los resultados de inferencia en una interfaz web (parte 1). 47
4.22. Visualización de los resultados de inferencia en una interfaz web (parte 2). 48

Índice de tablas

2.1. Atributos representativos 7

3.1. Desglose del espacio de variables 16
3.2. Distribución porcentual de speechiness antes y después del preprocesamiento 22
3.3. Distribución porcentual de liveness antes y después del preprocesamiento. 23
3.4. Resumen de parámetros usados en la selección de variables. 26
3.5. Resumen de hiperparámetros óptimos por objetivo. 30

4.1. Comparativa de métricas de dos experimentos de la red neuronal MLP. 34
4.2. Métricas de Essentia y del modelo de key y mode. 38
4.3. Resultados de la estimación de tempo de Essentia. 40
4.4. Métricas en el modelado de speechiness y liveness. 40

y compacta. El algoritmo de extracción, es el resultado de pruebas con distintas configuraciones de variables, estadísticos y métodos de segmentación. En el capítulo 4 se presentan los resultados del conjunto de variables predictivas más importantes por variable objetivo.

3.1.3. Definición del espacio de variables

El procesamiento computacional sobre el catálogo de canciones requirió más de una semana de ejecución para procesar y extraer la información de más de 60 000 canciones completas. El alto costo computacional se justificó por la necesidad de garantizar la disponibilidad de información para los experimentos de modelos predictivos de múltiples características musicales.

La aplicación del algoritmo final generó un conjunto de datos masivo compuesto por 9590 variables, resumido en la tabla 3.1.

TABLA 3.1. Desglose estructural de las 9590 variables extraídas por canción.

Categoría de extracción	Atributos base	Cant.	Estadísticos aplicados	Cant.	Total
Representaciones DSP (Librosa)	MFCC (20), Deltas (40) [17], Centroide, Ancho de banda, Rolloff, Contraste (7) [32], Flatness y Flux [32], Chroma (12), Tonnetz (6), RMS, ZCR.	92	Media, Desv. Estándar, Asimetría, Curtosis, Percentiles (10, 50, 90).	7	644
Atributos globales y ritmo	LUFS [35], Entropía espectral [17], Tempo, Complejidad dinámica [17], Tasa y estadísticos de onset (4) [16], Pitch, Fracción de voz [36], Claridad tonal (2) [16], Factor de cresta [32], Desviación de latidos [16].	14	Cálculo único por pista completa.	1	14
Estimación tonal (Perfiles)	Vectores de correlación Krumhansl (Mayor/Menor) y Temperley (Mayor/Menor) [16].	48	Cálculo único por pista completa (4 perfiles × 12 semitonos).	1	48
Embeddings semánticos (PANNs)	Vector latente de la capa previa a la clasificación de la red CNN14.	2048	Media, Máximo, Desv. Estándar, Percentil 90.	4	8 192
Probabilidades (PANNs)	Clases filtradas (Voz, Instrumentos, Géneros, Público en vivo, Distorsión).	173	Media, Máximo, Desv. Estándar, Percentil 90.	4	692
		Total de variables por canción:			9 590

3.2. Ejecución de tareas sobre el conjunto de datos

Esta sección presenta las variables objetivo y sus distribuciones en el contexto de Plusyc, el tratamiento aplicado a los datos y las técnicas de selección y reducción de variables predictivas.

3.2. Ejecución de tareas sobre el conjunto de datos

Esta sección presenta las variables objetivo y sus distribuciones en el contexto de Plusyc, el tratamiento aplicado a los datos y las técnicas de selección y reducción de variables predictivas.

3.2.1. Distribución y tratamiento de las variables objetivo

Durante la extracción de variables descrita en el algoritmo de la sección 3.1.2, se aplicaron dos estrategias adicionales para el tratamiento de los datos previos al modelado:

- Recorte de silencios y manejo de valores nulos: tras la primera iteración de extracción de variables (DSP), se detectó la presencia de valores nulos y ceros en algunas variables. El análisis exploratorio reveló que eran generados por las colas de silencio o fade-outs prolongados al final de las pistas. Para corregirlo, se aplicó un algoritmo de recorte de segmentos con amplitud inferior a -60 dB. Esta operación eliminó la gran mayoría de los nulos. El restante se concentró exclusivamente en un 6 % del descriptor `spectral_entropy_global`. Matemáticamente, el cálculo de la entropía espectral requiere normalizar la magnitud del espectro para interpretarlo como una distribución de probabilidad. En ventanas de análisis correspondientes a silencios absolutos, la energía espectral es nula [17].
- Filtrado del etiquetado manual: al analizar las variables objetivo, se identificó un cambio en una fecha específica en la fuente de los datos: hasta mediados de diciembre de 2024, los valores provenían de la integración con una API externa. Los registros generados durante 2025, ingresados manualmente por el equipo de curadores, introdujeron error, caracterizado por la presencia de valores por fuera del rango en variables discretas, valores extremos en cero y uno en variables continuas. Para proteger el entrenamiento de los modelos, el conjunto de datos de análisis fue truncado para conservar únicamente los registros fiables hasta diciembre de 2024. Resulta en el uso de un 92,5 % aproximado de las canciones del catálogo de Plusyc y aplica para todas las variables, excepto para géneros y estados de ánimo donde la información no dependía de la API.

A continuación, se presentan las distribuciones de las variables objetivo después del filtrado y el preprocesamiento adicional aplicado.

Loudness

El *loudness* define el volumen global promedio de la pista acústica en decibeles (dB) (ej. -5,2 dB o -12,4 dB). Los valores más negativos indican menor intensidad [11]. La figura 3.3 muestra la distribución de frecuencia de los valores del *loudness*. Se observa que presenta un sesgo negativo, con una concentración principal cerca de los -7 dB. Esto se explica dada la guerra del *loudness* [37], donde muchos artistas y productores buscan que sus canciones suenen con volumen alto, y plataformas como Spotify normalizan el *loudness* para evitar cambios abruptos en el volumen entre las canciones.

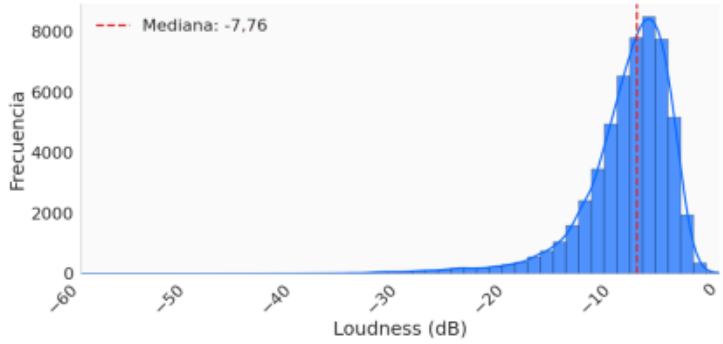


FIGURA 3.3. Distribución de *loudness* en el catálogo de Plusyc.

Se observan dos desafíos principales inherentes a la distribución:

- Existencia de asimetría negativa (sesgo a la izquierda).
- Presencia de valores extremos negativos (hasta -60 dB).

Esto potencialmente podría generar un error matemático que desvíe la atención del algoritmo durante el entrenamiento [24]. Se aplicaron las siguientes transformaciones como parte del preprocesamiento:

- Se aplicó una transformación logarítmica desplazada (Shifted Log-Transformation). Primero, se realizó una traslación de la escala $(y + 60,1)$ para garantizar que todos los registros fueran estrictamente mayores a cero.
- Posteriormente, se aplicó la función matemática $\log(1 + x)$. Este preprocesamiento comprime la cola izquierda de la distribución, mitiga el sesgo original y estabiliza la varianza para mejorar el rendimiento del modelo de regresión [24].

Key y mode

Las variables *key* y *mode* representan la tonalidad predominante de la pista mapeada a clases de tono enteras (ej. 0 para Do, 1 para Do#) y su modalidad armónica (1 para Mayor, 0 para Menor) [37]. La figura 3.4 muestra la distribución conjunta de las frecuencias de los valores discretos de las 24 tonalidades de la música occidental. Se observa preferencia por tonalidades como Do mayor, Re mayor y Sol mayor; y menor frecuencia de canciones en Re sostenido mayor y menor.

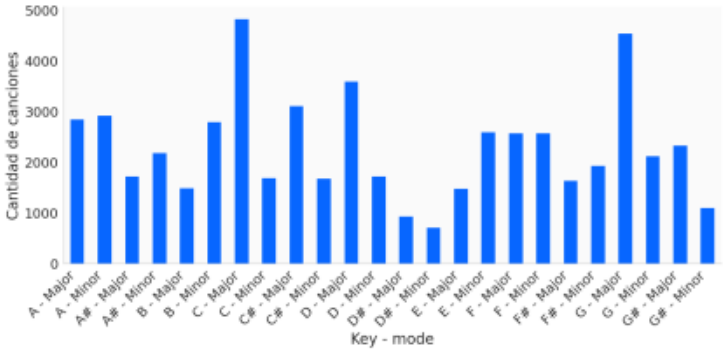


FIGURA 3.4. Distribución conjunta de *key* y *mode* en el catálogo de Plusyc.

Algunos desafíos inherentes a la distribución:

- Existe un desbalance marcado en las tonalidades, lo que propicia que los modelos de clasificación colapsen hacia las clases mayoritarias [24].
- Los algoritmos de aprendizaje automático requieren que las variables categóricas (como las notas musicales) estén representadas en un formato numérico procesable.

Se siguieron las siguientes estrategias como parte del preprocesamiento:

- Para adaptar las etiquetas alfanuméricas de las tonalidades al entrenamiento, se aplicó codificación de etiquetas (label encoding).
- Para mitigar el desbalance sin eliminar información musical intrínseca, el problema se abordó directamente en la fase de modelado mediante la ponderación de clases (`class_weights='balanced'`).

Se decide abordar el problema de clasificación conjunto de *key* y *mode* como una variable categórica con 24 clases (12 tonalidades × 2 modos). El motivo de la decisión se basa en que el caso de equivocación, o falso *mode*, es más problemático que el de *key* y *mode*, ya que afecta la familia armónica. La predicción de un top-k de *key* y *mode* conjuntos, generalmente muestran relaciones armónicas de la canción, esto tiene sentido en el contexto de la ambientación.

Tempo

El *tempo* indica la velocidad global estimada de la pista en pulsaciones por minuto (ej. 120 BPM u 85 BPM) [17]. La figura 3.5 muestra la distribución de frecuencia de los valores del *tempo*. Se observa una concentración principal en el rango de los 90 a 130 BPM, con un pico destacado alrededor de los 120 BPM, predominante en la música popular contemporanea [16].

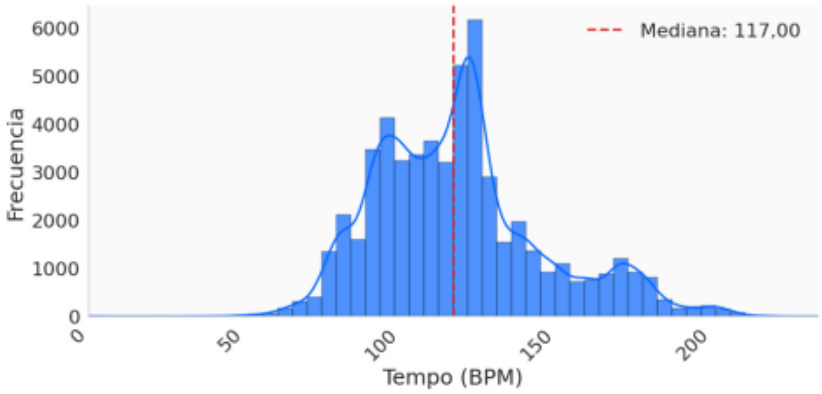


FIGURA 3.5. Distribución de *tempo* en el catálogo de Plusyc.

Se encuentra que *tempo* es un caso especial ya que presenta errores de octava métrica:

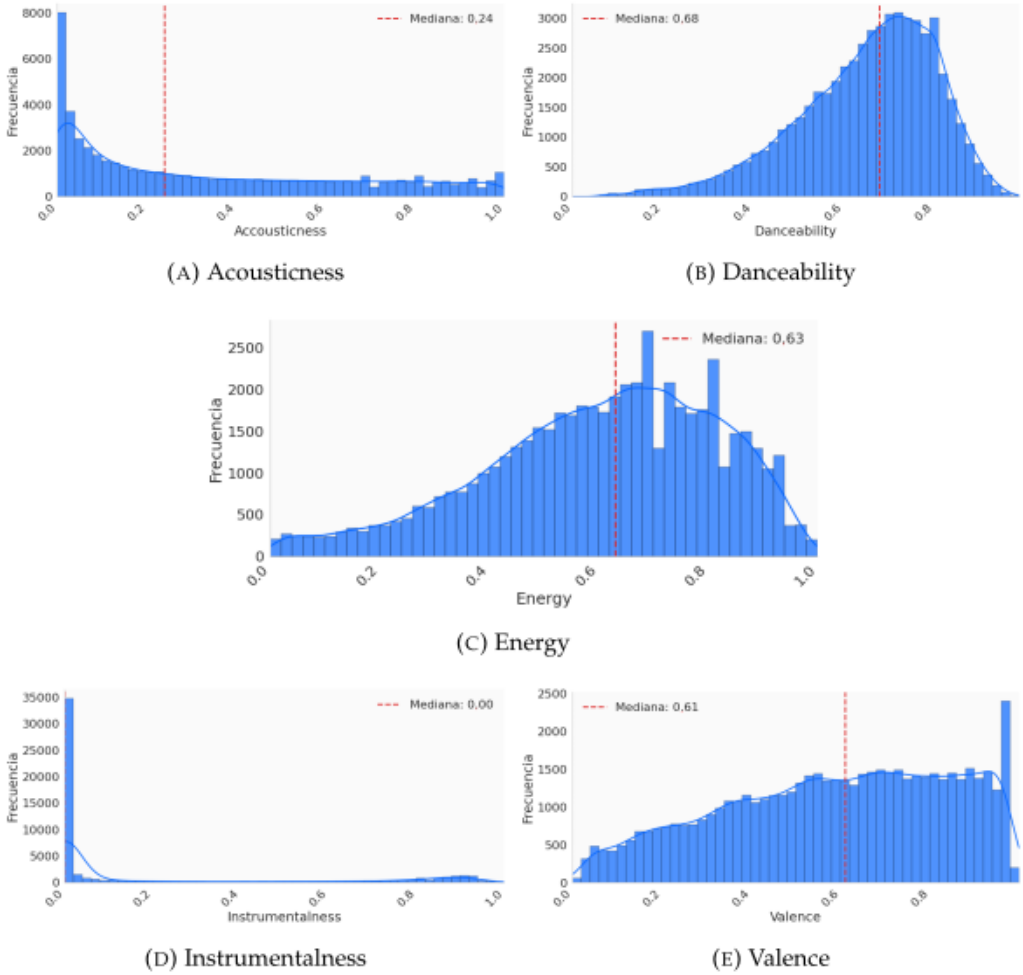
- Se identificaron inconsistencias en las etiquetas, donde el valor objetivo registrado representaba exactamente la mitad, el doble o proporciones fraccionarias (ej. 1,5x) del tempo real. Este es un problema documentado en la extracción de información musical conocido como error de octava (octave error) [16].

- Este problema puede explicar la presencia de valores atípicos en los extremos de la distribución.
- La presencia de errores de octava complica el uso de algoritmos de regresión lineal ya que se observan múltiples relaciones lineales entre variables rítmicas predictivas y el tempo global (ej. predecir 60 BPM para una pista etiquetada en 120 BPM).

Se contempla la evaluación de los algoritmos DSP de estimación de *tempo* integrados en las bibliotecas de DSP, esta comparación se muestra en la sección 4.1.4 del capítulo siguiente.

Acousticness, danceability, energy, instrumentalness y valence

Este grupo comprende variables perceptibles continuas evaluadas en el rango $[0, 1]$: *acousticness*: confianza acústica, *danceability*: bailaibilidad métrica, *energy*: intensidad enérgica, *instrumentalness*: ausencia vocal y *valence*: positividad musical [37]. La figura 3.6 muestra las distribuciones de frecuencia de los valores de estas características.



(A) Acousticness (B) Danceability

(C) Energy

(D) Instrumentalness (E) Valence

FIGURA 3.6. Distribuciones de características perceptibles en el catálogo de Plusyc.

Aunque todas comparten un rango de $[0, 1]$, presentan comportamientos distintos: *danceability* muestra una distribución aproximadamente normal; *energy* presenta sesgo negativo; *valence* tiende a la uniformidad; mientras que *acousticness* y de forma extrema, *instrumentalness*, exhiben un sesgo positivo con una concentración masiva de valores cercanos a cero.

El modelado conjunto de múltiples características se aborda convencionalmente mediante redes neuronales, como el Perceptrón Multicapa (MLP), dada su capacidad para aprender de forma simultánea las relaciones complejas y dependencias no lineales entre ellas [24]. Bajo esta premisa, el preprocesamiento requiere prestar atención específica en:

- La heterogeneidad en las formas de las distribuciones y sus varianzas. En arquitecturas sensibles a la escala, como los MLP, las variables con mayor varianza empírica pueden dominar el cálculo de los gradientes, ralentizar o sesgar la convergencia de los pesos [24].
- Las MLP son sensibles a los valores nulos y están presentes en un 6 % de `spectral_entropy_global`.

Se implementaron las siguientes medidas:

- A diferencia del *loudness*, las variables objetivo ya se encuentran acotadas matemáticamente en el intervalo $[0, 1]$. Por tanto, no requirieron transformaciones, ya que este rango es compatible con las funciones de activación continuas, como la sigmoid, en la capa de salida de la red neuronal.
- Se garantizó la integridad de la matriz de características al imputar los nulos residuales con el valor cero y así reflejar la falta de información acústica.
- Finalmente, se aplica estandarización estadística (StandardScaler) a todo el conjunto de entrada para garantizar una media de cero y varianza unitaria ($\mu = 0, \sigma^2 = 1$). Este paso es necesario para homogeneizar el espacio de variables predictivas para el entrenamiento de la red neuronal [24].

Speechiness

El *speechiness* detecta la presencia de palabras habladas en la pista y se define en $[0, 1]$, donde valores altos indican contenido exclusivamente hablado y valores bajos indican música (ej. 0,85 para conversación, 0,05 para música y 0,5 es rap) [17]. La figura 3.7 muestra la distribución de frecuencia del *speechiness*.

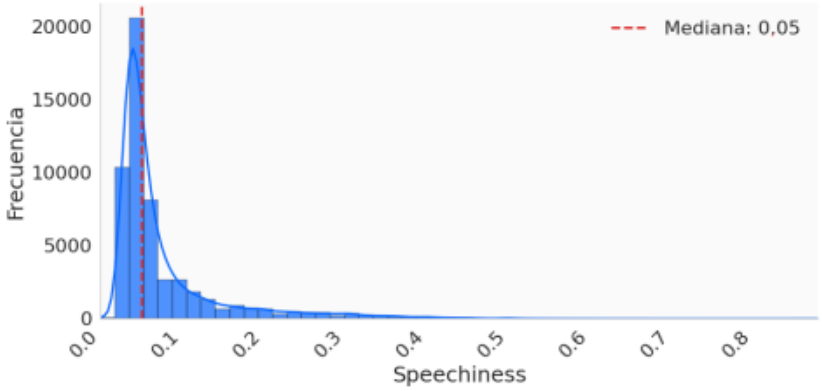


FIGURA 3.7. Distribución de *speechiness* en el catálogo de Plusyc.

La distribución muestra la presencia de un sesgo positivo extremo, ya que el catálogo musical carece de pistas de palabra hablada pura y concentra los valores muy cerca de cero. Los valores cerca del uno indican contenido exclusivamente hablado como pódcast o streaming conversacional, ausentes en el catálogo de Plusyc.

El mejor resultado experimental en términos de las métricas de evaluación se obtuvo al aplicar técnicas de submuestreo. Se discretizó el dominio de la variable en rangos estratégicos y se aplicó un submuestreo con tope por rangos para limitar la cantidad de registros en la moda, reducir el sesgo y conservar la totalidad de registros de la cola. Los rangos se presentan en la tabla 3.2.

TABLA 3.2. Distribución porcentual de speechiness antes y después del preprocesamiento

Intervalo	Descripción del rango	% Original	% Final
$[0,00, 0,05]$	Menor presencia de canto	56,0 %	42,1 %
$(0,05, 0,33]$	Canto	41,9 %	42,1 %
$(0,33, 1,00]$	Rap	2,1 %	15,7 %
Total		100 %	100 %

Cabe mencionar que se aplicó muestreo estratificado a nivel de los rangos durante la división de conjuntos de entrenamiento y pruebas. Finalmente, se mitigó el ruido en las etiquetas al eliminar un porcentaje menor de registros contradictorios: se compararon los valores de la variable objetivo con la evidencia acústica vocal en las probabilidades de clases como *speech* y *rapping* extraídas de PANNs para descartar las pistas que presentaban discrepancias entre su etiqueta manual y su contenido semántico real.

Liveness

El *liveness* representa la confianza de que la pista fue grabada con público en vivo $[0, 1]$, (ej. 0,85 para un concierto, 0,10 para un estudio) [17]. La figura 3.8 muestra la distribución de frecuencia del *liveness*.

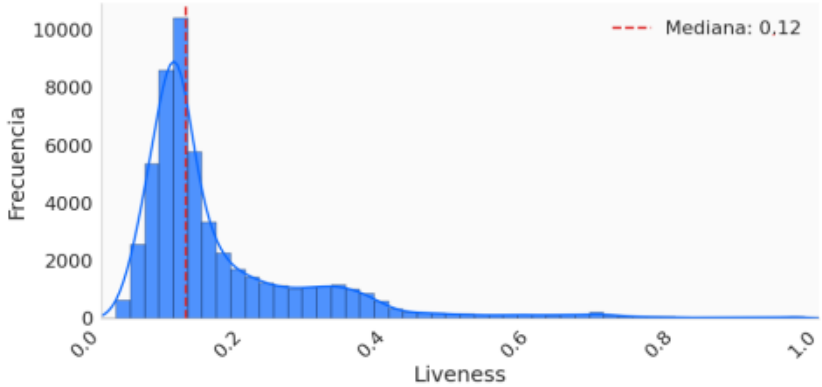


FIGURA 3.8. Distribución de *liveness* en el catálogo de Plusyc.

Esta distribución, como en el caso anterior, también muestra la presencia de un sesgo positivo extremo, ya que concentra la inmensa mayoría de los registros por debajo de 0,2. Adicionalmente, se identificó un problema de ruido sistemático

en las etiquetas manuales, donde pistas de estudio con una alta percepción de energía o intensidad eran catalogadas erróneamente como "vivo".

Se aplicó una técnica similar a la anterior para balancear el objetivo continuo, se dividió el dominio con base en la definición teórica de la característica y se aplicó un submuestreo con tope por rangos. Se aplicó reducción en la clase mayoritaria y en la zona intermedia para evitar sesgos y se conservan todos los registros minoritarios de la clase de interés: canción en vivo. El resultado de esta reestructuración se detalla en la tabla 3.3.

TABLA 3.3. Distribución porcentual de liveness antes y después del preprocesamiento.

Intervalo	Descripción del rango	% Original	% Final
$[0,0,0,2)$	Muy baja probabilidad de vivo	74,4 %	67,4 %
$[0,2,0,8)$	Baja probabilidad de vivo	24,8 %	22,5 %
$[0,8,1,0]$	Alta probabilidad de vivo	0,8 %	10,1 %
Total		100 %	100 %

Como en el caso de *speechiness*, también se aplica muestreo estratificado por rangos durante la separación de conjuntos de entrenamiento y pruebas. Y se implementó un filtrado cruzando la variable objetivo con la evidencia de presencia de audiencia extraída por la red PANNs, de las probabilidades de clases asociadas a multitudes, aplausos y aclamaciones. Se descartaron las pistas que presentaban discrepancias severas para eliminar ruido (ej. registros etiquetados como 'en vivo' sin evidencia de sonido de público, o viceversa).

Top-5 primary mood

El *primary mood* define el estado de ánimo o emoción predominante que transmite la pista (ej. *Energizing, Lively, Empowering*). La figura 3.9 muestra los *primary mood* de Plusyc y sus frecuencias.

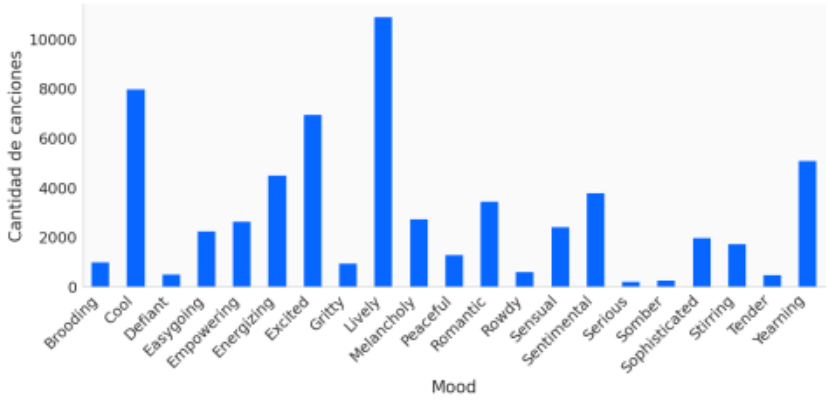


FIGURA 3.9. Distribución de *primary mood* en el catálogo de Plusyc.

Se observan los siguientes desafíos en esta variable categórica:

- Alta cardinalidad y un marcado desbalance de clases, donde ciertas emociones predominan debido a la orientación comercial y de ambientación del catálogo. Algunas clases se han dejado de usar en el pasado.

3.2.2. Selección de variables de entrada

La aplicación del algoritmo de extracción generó un espacio predictivo masivo compuesto por 9590 variables. Modelar directamente sobre esta dimensionalidad introduce riesgo de sobreajuste y alto costo computacional [38, 39]. Por ello, la estrategia de diseño trasladó el esfuerzo hacia el proceso de selección de variables predictivas.

Para todas las variables objetivo, el proceso de selección se ejecutó estrictamente sobre los datos de entrenamiento (X_{train}) para evitar srobreajuste, mediante un flujo secuencial de tres etapas:

1. Filtro por varianza: se implementó `VarianceThreshold` [40] para evaluar cada característica de forma aislada y descartar aquellas constantes o sin varianza (umbral 0,00). Se aplicó un umbral de 0,01 en objetivos de alta cardinalidad para purgar características dispersas que carecen de valor predictivo [41].
2. Selección basada en árboles, con las siguientes características:
   - El cálculo de la importancia se realizó predominantemente mediante un entrenamiento simple, para priorizar la eficiencia computacional. Excepcionalmente, en las variables perceptibles continuas, se implementó validación cruzada (K-Fold con $K = 5$) para mitigar el sobreajuste al ruido en este tipo de etiquetas subjetivas [42].
   - La métrica utilizada fue la ganancia (gain), para medir el aporte de las variables en las particiones de los árboles [43, 22].
   - Se conservaron únicamente las características con mayor importancia, responsables de explicar el 95 % de la ganancia acumulada del modelo.
3. Filtro por dependencia lineal: se calculó la matriz de correlación de Pearson para analizar las características en conjunto y eliminar aquellas que presentaban redundancia lineal [44, 41].

En la tabla 3.4 se resume la configuración del proceso y el total de variables predictivas resultantes para cada variable objetivo.

TABLA 3.4. Resumen de parámetros usados en la selección de variables.

Variable	Varianza	Validación	Colinealidad	Total
Key y *mode*	0,00	Simple	–	173
Loudness	0,00	Simple	–	856
Perceptibles (5)	0,00	K-Fold ($K = 5$)	$> 0,95$	2 145
Speechiness	0,00	Simple	$> 0,95$	92
Liveness	0,00	K-Fold ($K = 5$)	$> 0,95$	67
Primary mood	0,01	Simple	$> 0,95$	1 423
Secondary mood	0,01	Simple	$> 0,95$	1 397
Suggested genres	0,01	Simple	$> 0,95$	1 309

La columna "Varianza", indica el umbral del filtro por varianza, "Validación", indica el tipo de validación utilizada (Simple o K-Fold), y "Colinealidad", indica el umbral de correlación para el filtro por dependencia lineal.

3.3. Modelado y optimización

Modelado mediante perceptrón multicapa (MLP)

Para la predicción de las cinco variables perceptibles (*acousticness, danceability, energy, instrumentalness, valence*), se implementó una red neuronal de tipo perceptrón multicapa. La arquitectura diseñada consta de tres capas ocultas con una reducción progresiva de neuronas (512, 256 y 128 neuronas, respectivamente). La capa de salida cuenta con cinco neuronas y emplea una función de activación sigmoid, para garantizar que las predicciones se acoten dentro del rango físico [0, 1]. La figura 3.12 ilustra la arquitectura general del perceptrón multicapa implementado.

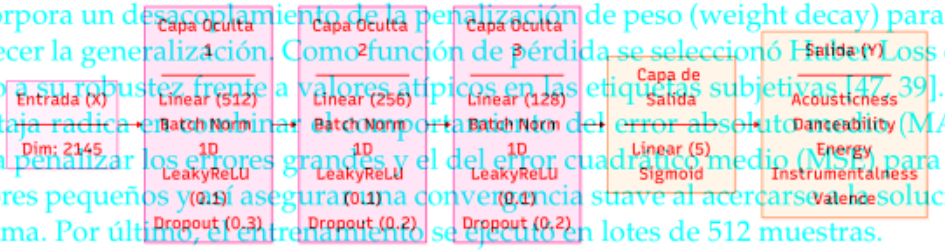


FIGURA 3.12. Bloques del perceptrón multicapa para las variables perceptibles.

Para la optimización de pesos en el entrenamiento se empleó el algoritmo AdamW [45]. Este método incorpora un desacoplamiento de la penalización de peso (weight decay) para favorecer la generalización. Como función de pérdida se seleccionó Huber Loss debido a su robustez frente a valores atípicos en las etiquetas subjetivas [46, 38]. Su ventaja radica en combinar el comportamiento del error absoluto medio (MAE) para penalizar los errores grandes y el del error cuadrático medio (MSE) para los errores pequeños y así asegurar una convergencia suave al acercarse a la solución óptima. Por último, el entrenamiento se ejecutó en lotes de 512 muestras.

Para el ajuste de los hiperparámetros de aprendizaje, la tasa inicial se fijó en 10^{-4} y fue gestionada dinámicamente mediante `ReduceLROnPlateau`. Este mecanismo redujo la tasa de aprendizaje a la mitad automáticamente al detectar estancamientos.

Dado el amplio espacio de variables (2 145 variables de entrada) y la alta capacidad del modelo, se implementó la siguiente estrategia de regularización para mitigar el sobreajuste [24]:

- Capas de normalización por lotes (batch normalization) después de cada transformación lineal ($Wx + b$) en `nn.Linear`, para evitar que los valores numéricos se disparen entre las capas y facilitar la convergencia [47].
- Inactivación aleatoria de neuronas (dropout) con tasas de 0,30 en la primera capa oculta y 0,20 en las subsiguientes, como filtro más agresivo de ruido

en las variables iniciales y proteger las representaciones más abstractas en las capas profundas [48].

- Penalización L2 (regularización de Tikhonov) integrada en el optimizador con un factor de 10^{-5}.
- Parada anticipada (early stopping) basada en el conjunto de validación que interrumpió el entrenamiento tras 15 iteraciones sin mejoras.

El diseño de los modelos se adaptó a la naturaleza de cada variable objetivo, con base en el preprocesamiento detallado en etapas anteriores:

- *Key / mode, primary, secondary moods y suggested genres*: mayor peso a las clases minoritarias mediante el parámetro `class_weight='balanced'` y se evaluó el desempeño con la métrica F1-Score macro.
- *Liveness, speechiness*: la separación de los conjuntos de entrenamiento y validación se ejecutó de forma estratificada tras discretizar las variables en rangos probabilísticos.
- *Loudness*: la optimización evaluó el MAE, y el modelado definitivo se configuró mediante regresión estándar (L2) para minimizar el error cuadrático medio (RMSE). Esto demostró una mejora en la predicción en volúmenes extremos.

Optimización de hiperparámetros y regularización

Se implementó una optimización bayesiana (`BayesianOptimization`) para el objetivo multiclase (`multi_logloss`) [23], ajustando parámetros de árboles y regularización relacionadas:

- *Loudness*: la optimización evaluó el MAE, y el modelado definitivo se configuró mediante regresión estándar (L2) para minimizar el error cuadrático medio (RMSE). Esto demostró una mejora en la predicción en volúmenes extremos como se puede ver en la sección 4.1.2.

La figura 3.13 resume lo anterior.

- Submuestreo de filas y columnas para mejorar la generalización. En la mayoría de los modelos se utilizó el algoritmo (`GBDT`) combinado con un submuestreo de filas aleatorio (`subsample`). Excepto para *loudness*, donde se implementó el algoritmo de muestreo basado en gradientes (`GOSS`) [22], para retener ejemplos con mayor error predictivo y descartar una fracción de las que el modelo ya aprendió.
- Penalizaciones (L1 y L2) para frenar el peso excesivo de las ramas [39]. Adicionalmente, en la mayoría de los objetivos, se exigió una mejora mínima (`min_split_gain`) antes de permitir que el árbol se dividiera.
- Early stopping, configurado entre 30 y 50 iteraciones, para interrumpir el entrenamiento cuando el modelo dejara de mejorar.

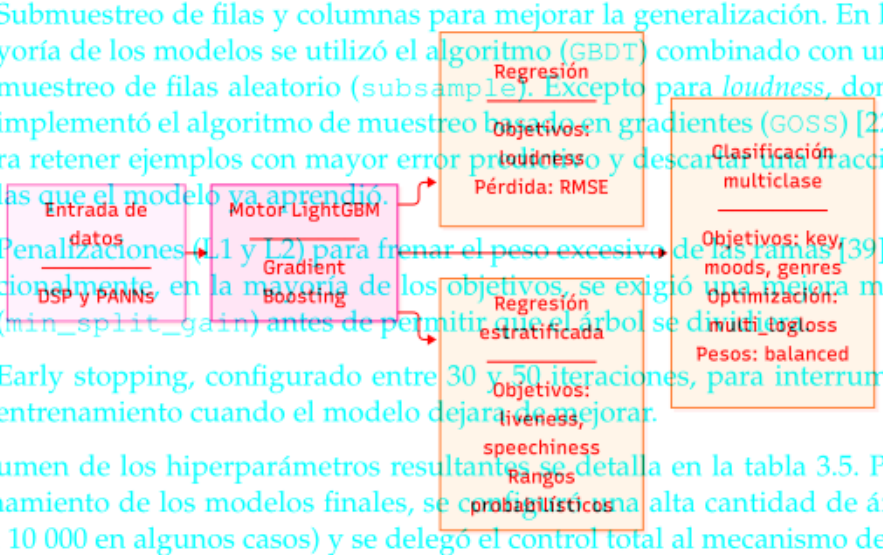


FIGURA 3.13: Resumen del modelado LightGBM.

El resumen de los hiperparámetros resultantes se detalla en la tabla 3.5. Para el entrenamiento de los modelos finales, se configuró una alta cantidad de árboles (hasta 10 000 en algunos casos) y se delegó el control total al mecanismo de early stopping. Esto, combinado con la optimización bayesiana, mejora el desempeño [39].

Optimización de hiperparámetros y regularización

Se implementó una búsqueda automatizada de hiperparámetros para cada objetivo mediante la biblioteca Optuna [23]. A continuación un resumen de los parámetros y algunas decisiones relacionadas:

- Profundidad máxima de los árboles (`max_depth`) y restricción en el número máximo de hojas (`num_leaves`) para evitar el crecimiento asimétrico descontrolado. Además, se exigió un volumen mínimo de muestras por hoja (`min_child_samples` o `min_data_in_leaf`) para evitar que los modelos crearan reglas exclusivas para registros aislados [22].
- Submuestreo de filas y columnas para mejorar la generalización. En la mayoría de los modelos se utilizó el algoritmo (`GBDT`) combinado con un submuestreo de filas aleatorio (`subsample`). Excepto para *loudness*, donde se implementó el algoritmo de muestreo basado en gradientes (`GOSS`) [22], para retener ejemplos con mayor error predictivo y descartar una fracción de las que el modelo ya aprendió.
- Penalizaciones (L1 y L2) para frenar el peso excesivo de las ramas [38]. Adicionalmente, en la mayoría de los objetivos, se exigió una mejora mínima (`min_split_gain`) antes de permitir que el árbol se dividiera.
- Early stopping, configurado entre 30 y 50 iteraciones, para interrumpir el entrenamiento cuando el modelo dejara de mejorar.

Una simplificación del flujo de optimización de parámetros se puede ver en la figura 3.14.

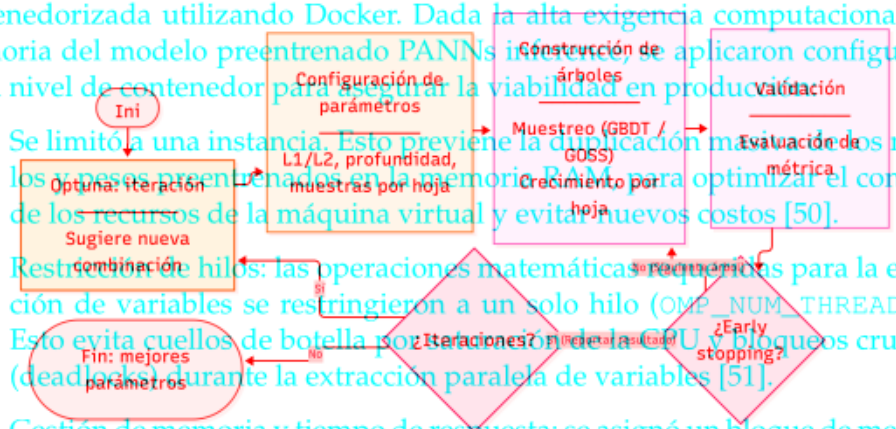


FIGURA 3.14. Flujo de optimización de parámetros para los modelos LightGBM.

El resumen de los hiperparámetros resultantes se detalla en la tabla 3.5. Para el entrenamiento de los modelos finales, se configuró una alta cantidad de árboles (hasta 10 000 en algunos casos) y se delegó el control total al mecanismo de early stopping. Esto, combinado con tasas de aprendizaje muy bajas, mejora el desempeño [38].

3.4. Arquitectura de despliegue e integración

El sistema de inferencia fue diseñado para operar como un servicio integrable con la infraestructura existente de Plusyc. El despliegue se realizó en una instancia de cómputo sobre OCI.

3.4.1. Diseño del servicio predictivo

El núcleo de procesamiento se expuso a través de una API RESTful desarrollada con FastAPI. Esta interfaz expone el ciclo de inferencia: carga del audio, extracción de variables predictivas de entrada a los modelos MLP y LightGBM, que toman decisiones de tipo predictivo sobre la conveniencia de la canción.

Para garantizar la reproducibilidad y estabilidad del entorno, la aplicación fue contenedorizada utilizando Docker. Dada la alta exigencia computacional y de memoria del modelo preentrenado PANNs inference, se aplicaron configuraciones a nivel de contenedor para asegurar la viabilidad en producción:

- Se limitó a una instancia. Esto previene la duplicación masiva de los modelos y pesos preentrenados en la memoria RAM para optimizar el consumo de los recursos de la máquina virtual y evitar nuevos costos [50].
- Restricción de hilos: las operaciones matemáticas requeridas para la extracción de variables se restringieron a un solo hilo (`OMP_NUM_THREADS=1`). Esto evita cuellos de botella por saturación de la CPU y bloqueos cruzados (deadlocks) durante la extracción paralela de variables [51].
- Gestión de memoria y tiempo de respuesta: se asignó un bloque de memoria compartida (`shm_size: '5gb'`) y se establecieron timeouts holgados (120 segundos) para prevenir interrupciones provocadas por el análisis de pistas de audio de larga duración, restringidas a 15 máximo máximo por canción.

3.4.2. Flujo de integración

La interacción del motor predictivo con el ecosistema web se realiza a través de un flujo de procesamiento asíncrono. La comunicación entre componentes es orquestada por una herramienta de Plusyc denominada Beethoven.

El ciclo de vida de la integración, ilustrado en la figura 3.12, sigue esta secuencia lógica:

TABLA 3.5. Resumen de hiperparámetros óptimos por objetivo.

Objetivo	Prof. máx.	Hojas	Tasa de apren.	Muestras/Hoja	L1 (α)	L2 (λ)
Key/Mode	5	140	0,0259	34	3,1690	6,9757
Loudness	10	21	0,0868	25	0,0104	0,0390
Liveness	4	15	0,0194	17	0,0107	0,3887
Speechiness	8	34	0,0102	32	0,4942	0,2947
Primary Mood	10	32	0,0208	79	0,1084	5,0239
Secondary Mood	11	35	0,0125	69	0,1443	2,6547
Genres	6	28	0,0606	21	0,9334	4,8052

1. Una tarea programada en el sistema ejecuta el CLI Beethoven a intervalos regulares de tiempo.
2. El proceso detecta nuevos archivos de audio que aún no han sido caracterizados.
3. Por cada nueva pista detectada, Beethoven envía el archivo al servicio de inferencia (API) mediante una petición HTTP POST.
4. La Inferencia: la API ejecuta el pipeline predictivo, que recibe la señal cruda y retorna un diccionario estructurado (JSON) que contiene todas las características estimadas (*key, moods, liveness, etc.*).
5. El script captura la respuesta de la API, extrae las características musicales y las persiste en la base de datos de información musical de Plusyc.
6. Una vez en la base de datos, los metadatos quedan disponibles para revisión de curadores en un portal web.

3.4. Arquitectura de despliegue e integración

El sistema de inferencia fue diseñado para operar como un servicio integrable con la infraestructura existente de Plusyc. El despliegue se realizó en una instancia de cómputo sobre OCI.

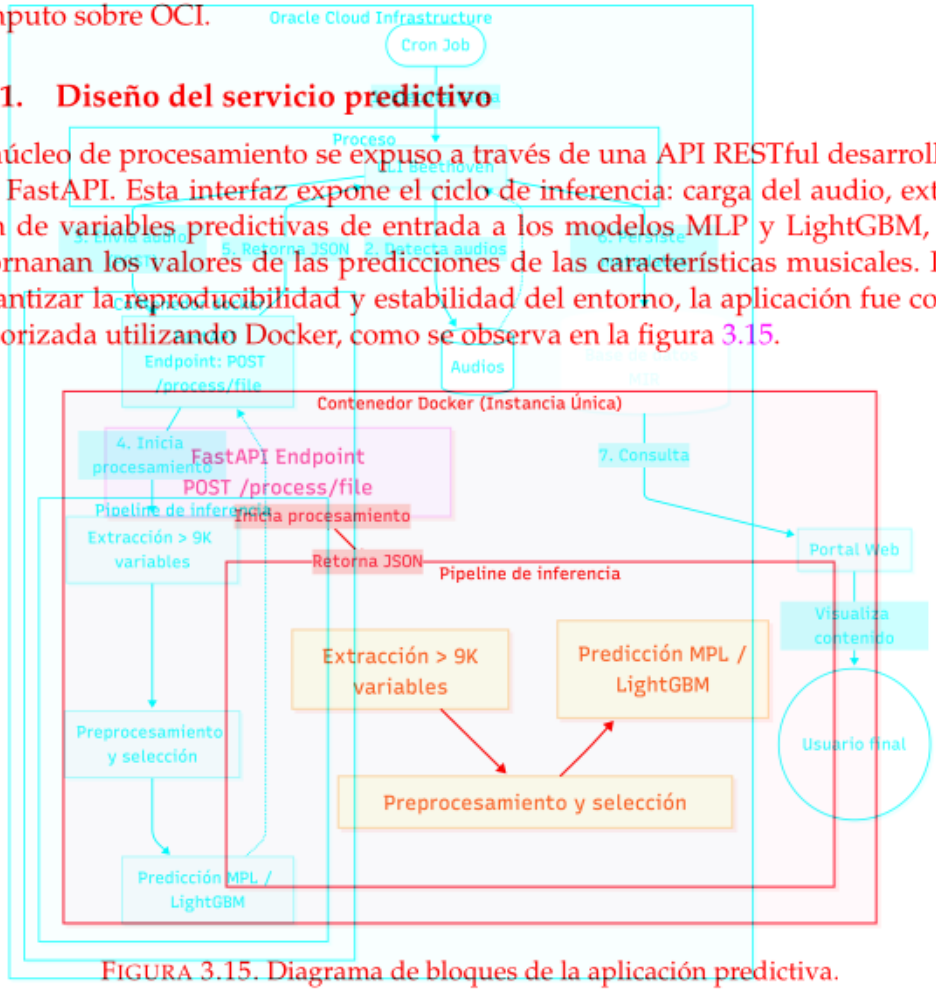


FIGURA 3.15. Diagrama de bloques de la aplicación predictiva.

FIGURA 3.12. Diagrama de despliegue e integración.

3.4.1. Diseño del servicio predictivo

El núcleo de procesamiento se expuso a través de una API RESTful desarrollada con FastAPI. Esta interfaz expone el ciclo de inferencia: carga del audio, extracción de variables predictivas de entrada a los modelos MLP y LightGBM, que retornanan los valores de las predicciones de las características musicales. Para garantizar la reproducibilidad y estabilidad del entorno, la aplicación fue contenedorizada utilizando Docker, como se observa en la figura 3.15.

Dada la alta exigencia computacional y de memoria del modelo preentrenado PANNs inference, se aplicaron configuraciones a nivel de contenedor para asegurar la viabilidad en producción:

- Se limitó a una instancia. Esto previene la duplicación masiva de los modelos y pesos preentrenados en la memoria RAM, para optimizar el consumo de los recursos de la máquina virtual y evitar nuevos costos [49].

- Restricción de hilos: las operaciones matemáticas requeridas para la extracción de variables se restringieron a un solo hilo (`OMP_NUM_THREADS=1`). Esto evita cuellos de botella por saturación de la CPU y bloqueos cruzados (deadlocks) durante la extracción paralela de variables [50].
- Gestión de memoria y tiempo de respuesta: se asignó un bloque de memoria compartida (`shm_size: '5gb'`) y se establecieron timeouts holgados (120 segundos) para prevenir interrupciones provocadas por el análisis de pistas de audio de larga duración, restringidas a 15 máximo máximo por canción.

3.4.2. Flujo de integración

La interacción del motor predictivo con el ecosistema web se realiza a través de un flujo de procesamiento asíncrono. La comunicación entre componentes es orquestada por una herramienta de Plusyc denominada Beethoven.

El ciclo de vida de la integración, ilustrado en la figura 3.16, sigue esta secuencia lógica:

1. Una tarea programada en el sistema ejecuta el CLI Beethoven a intervalos regulares de tiempo.
2. El proceso detecta nuevos archivos de audio que aún no han sido caracterizados.
3. Por cada nueva pista detectada, Beethoven envía el archivo al servicio de inferencia (API) mediante una petición HTTP POST.
4. Inferencia: la API ejecuta el pipeline predictivo, que recibe la señal cruda y rertorna un diccionario estructurado (JSON) que contiene todas las etiquetas estimadas *(key, moods, liveness, etc.)*.
5. El script captura la respuesta de la API, extrae las características musicales y las persiste en la base de datos de información musical de Plusyc.
6. Una vez en la base de datos, los metadatos quedan disponibles para revisión de curadores en un portal web.

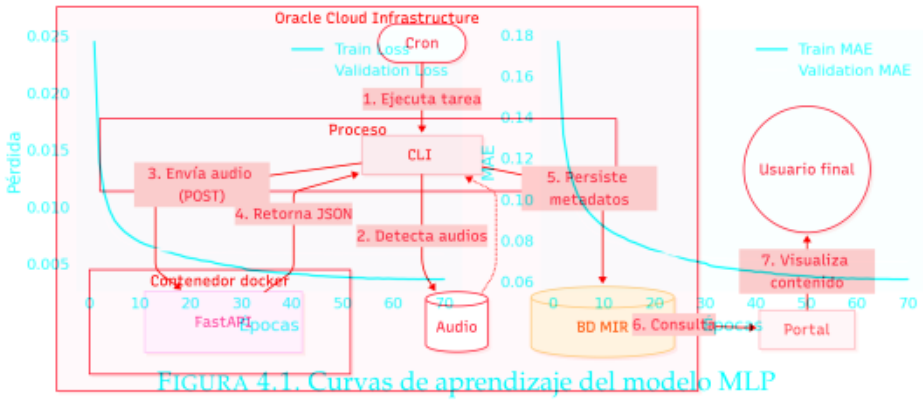


FIGURA 4.1. Curvas de aprendizaje del modelo MLP

FIGURA 3.16. Diagrama de despliegue e integración.

La evolución de la pérdida de Huber a la izquierda y el MAE a la derecha, demuestran una convergencia rápida y estable. En ambos casos, las curvas de validación y entrenamiento se cruzan y estabilizan alrededor de las iteraciones 12 y 20, respectivamente. A partir de este punto, la validación se mantiene constante

mientras el error de entrenamiento presenta un descenso marginal y evidencian aprendizaje sin sobreajuste.

La tabla 4.1 muestra cómo la inclusión de embeddings y probabilidades de PANNs inference y la separación de *speechiness* y *liveness* mejoró consistentemente todas las métricas, con R^2 global de 0,8152.

TABLA 4.1. Comparativa de métricas de dos experimentos de la red neuronal MLP.

Variable	**R^2 (↑)**		**MAE (↓)**		**MSE (↓)**	
	Inicial	**Definitivo**	**Inicial**	**Definitivo**	**Inicial**	**Definitivo**
Acousticness	0,8178	0,8903	0,0950	0,0681	0,0171	0,0106
Danceability	0,6042	0,7381	0,0780	0,0594	0,0108	0,0063
Energy	0,7584	0,8626	0,0809	0,0581	0,0110	0,0062
Instrumentalness	0,6733	0,8308	0,1231	0,0671	0,0355	0,0188
Valence	0,6523	0,7539	0,1172	0,0964	0,0221	0,0162
Liveness	0,1570	–	–	–	–	–
Speechiness	0,2778	–	–	–	–	–

La figura 4.2 muestra los resultados de los valores reales de las etiquetas y las estimaciones del modelo. En estos gráficos de dispersión, la línea diagonal de referencia representa el escenario de predicción perfecta. La proximidad y densidad de los puntos respecto a esta recta ilustran la relación lineal, la varianza y la capacidad del modelo frente a datos no vistos.

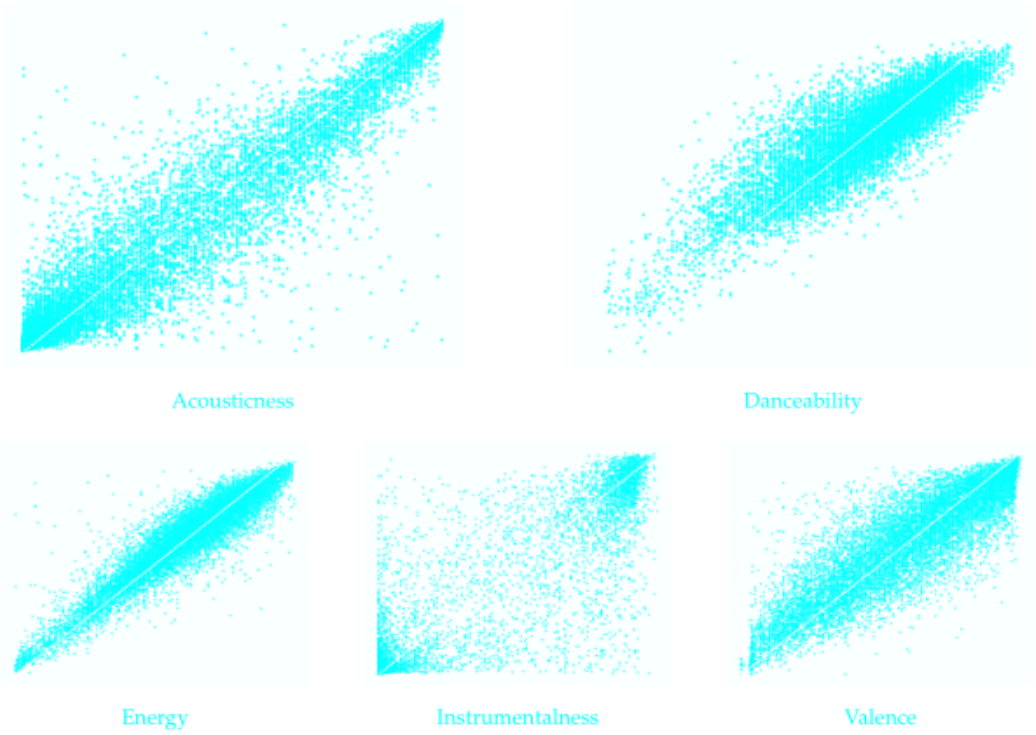


FIGURA 4.2. Relación de valores reales y estimados en el modelo MLP.

Capítulo 4

Ensayos y resultados

Este capítulo presenta los resultados de los modelos seleccionados para cada variable objetivo, su comparación con un experimento previo en casos pertinentes, los resultados de la evaluación con métricas de inteligencia artificial y un ejemplo de inferencia.

4.1. Resultados por modelo

A continuación se exponen los experimentos y las métricas de evaluación.

4.1.1. Acousticness, danceability, energy, instrumentalness y valence

En los primeros ensayos de inferencia de este grupo de variables objetivo, se entrenó un modelo multisalida únicamente con variables DSP para predecir siete variables en total: *acousticness*, *danceability*, *energy*, *instrumentalness*, *valence*, *liveness* y *speechiness*, todas en el rango de 0 a 1. Se obtuvo un rendimiento insuficiente, incapaz de predecir *liveness* y *speechiness*, pero con señal de aprendizaje para las demás variables objetivo. Así se identificó la necesidad de separar las dos variables, en su preprocesamiento e inferencia.

El diseño actual de la MLP incorpora las variables DSP, embeddings y probabilidades de PANNs inference. Durante el entrenamiento, el mecanismo de early stopping detuvo el proceso en la iteración 74, después de 15 épocas sin reducir el error de validación mínimo de 0,0058, alcanzado en la iteración 59. La figura 4.1 ilustra las curvas de aprendizaje del modelo durante el entrenamiento.

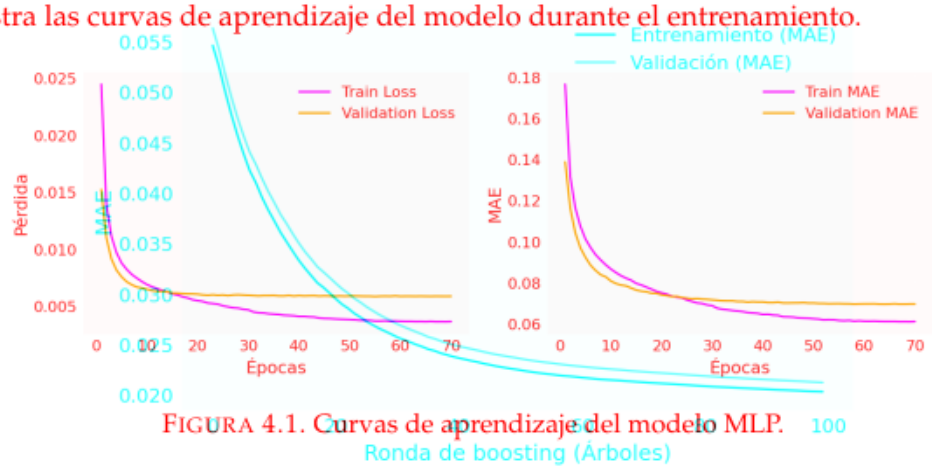


FIGURA 4.1. Curvas de aprendizaje del modelo MLP.

La evolución de la pérdida de Huber a la izquierda y el MAE a la derecha, demuestran una convergencia rápida y estable. En ambos casos, las curvas de validación y entrenamiento se cruzan y estabilizan alrededor de las iteraciones 12 y

20, respectivamente. A partir de este punto, la validación se mantiene constante mientras el error de entrenamiento presenta un descenso marginal y evidencian aprendizaje sin sobreajuste.

La tabla 4.1 muestra cómo la inclusión de embeddings y probabilidades de PANNs inference y la separación de *speechiness* y *liveness* mejoró consistentemente todas las métricas, con R^2 global de 0,8152.

TABLA 4.1. Comparativa de métricas de dos experimentos de la red neuronal MLP.

Variable	R^2 (↑)		MAE (↓)		MSE (↓)	
	Inicial	Definitivo	Inicial	Definitivo	Inicial	Definitivo
Acousticness	0,8178	0,8903	0,0950	0,0681	0,0171	0,0106
Danceability	0,6042	0,7381	0,0780	0,0594	0,0108	0,0063
Energy	0,7584	0,8626	0,0809	0,0581	0,0110	0,0062
Instrumentalness	0,6733	0,8308	0,1231	0,0671	0,0355	0,0188
Valence	0,6523	0,7539	0,1172	0,0964	0,0221	0,0162
Liveness	0,1570	–	–	–	–	–
Speechiness	0,2778	–	–	–	–	–

La figura 4.2 muestra los resultados de los valores reales de las etiquetas y las estimaciones del modelo. En estos gráficos de dispersión, la línea diagonal de referencia representa el escenario de predicción perfecta. La proximidad y densidad de los puntos respecto a esta recta ilustran la relación lineal, la varianza y la capacidad del modelo frente a datos no vistos.

La figura 4.6 detalla las 15 variables predictivas principales. El modelo prioriza el factor de cresta y los LUFS, seguidos de estadísticos de las representaciones DSP tradicionales como el RMS y el Centroide espectral. A diferencia de las variables perceptibles analizadas en la sección anterior, en este modelo sí destaca la presencia de un embedding de PANNs inference dentro del conjunto de mayor relevancia.

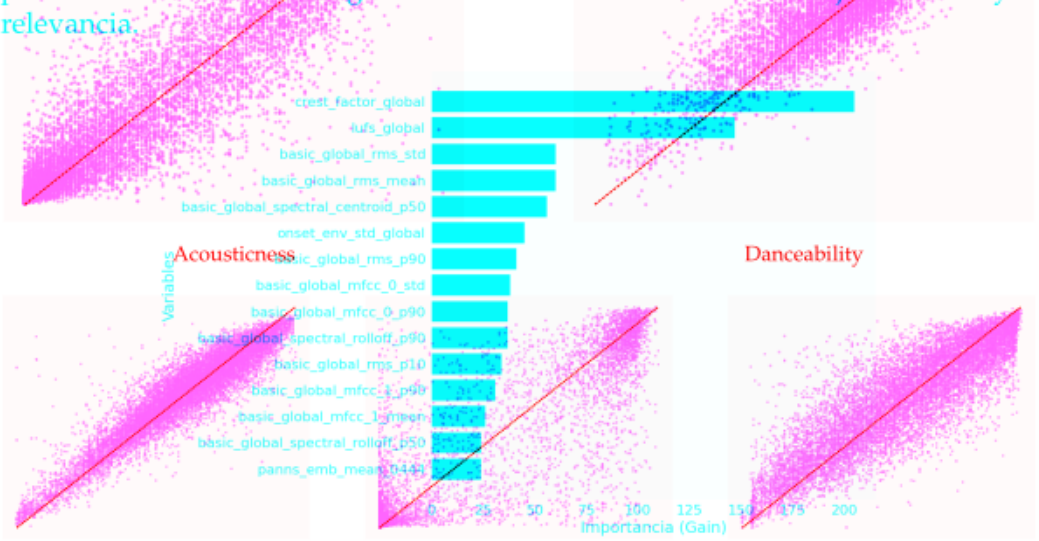


FIGURA 4.6. Top-15 variables predictivas para loudness.

FIGURA 4.2. Relación de valores reales y estimados en el modelo MLP.

4.1.3. Key y mode

Una lista de importancia de variables predictivas muestra predominio de las variables globales y ritmo presentadas en la tabla 3.1. La figura 4.3 presenta las 15 variables más relevantes del modelo MLP. En la figura, el grado de importancia equivale a la caída en el coeficiente de determinación (R^2) que sufre el modelo al eliminar la información de dicha variable. Se destaca el impacto principal del factor de cresta, representaciones chroma y estadísticos de onset. También es interesante la ausencia de embeddings y probabilidades de PANNs en el top-15, pero muchas están presente entre las variables seleccionadas (2 145 en total), lo que muestra que operan en conjunto.

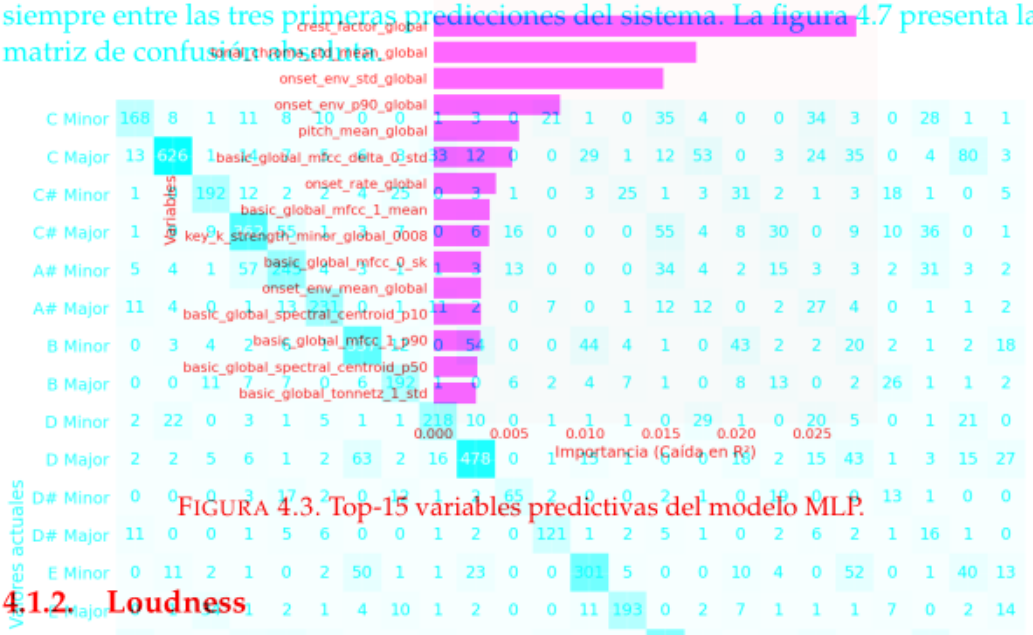


FIGURA 4.3. Top-15 variables predictivas del modelo MLP.

4.1.2. Loudness

La inferencia de *loudness* consistió en un modelo de regresión LightGBM. En la configuración inicial se probó con penalización L1 (`regression_l1`). La figura 4.4 expone las curvas de aprendizaje para este experimento, donde se observa el descenso de los errores de entrenamiento y validación a medida que se suman árboles al modelo. La trayectoria y estabilización conjunta de ambas curvas evidencian un aprendizaje estable.

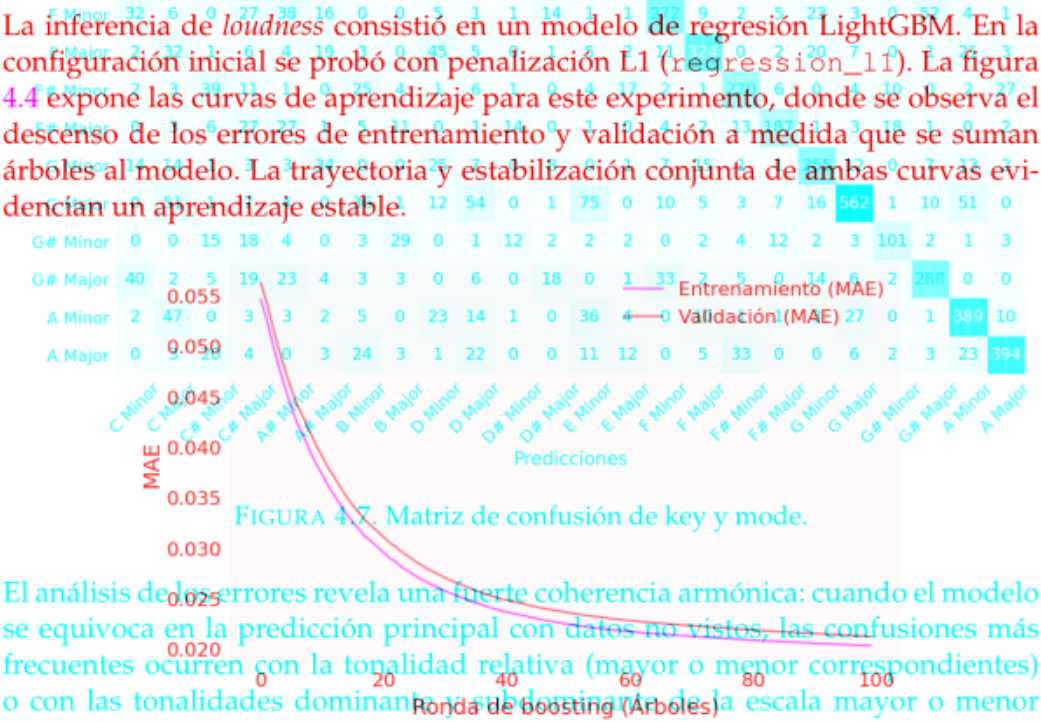


FIGURA 4.7. Matriz de confusión de key y mode.

El análisis de los errores revela una fuerte coherencia armónica: cuando el modelo se equivoca en la predicción principal con datos no vistos, las confusiones más frecuentes ocurren con la tonalidad relativa (mayor o menor correspondientes) o con las tonalidades dominante y subdominante de la escala mayor o menor correspondiente.

FIGURA 4.4. Curvas de aprendizaje para loudness.

Para contextualizar este desempeño, la tabla 4.2 contrasta los resultados del extractor de tonalidad de la librería Essentia evaluado en el dataset de Plusyc, con

Esta primera configuración alcanzó un R^2 de 0,8300, un RMSE de 1,8534 y un MAE de 1,1469. Es un buen modelo, con leve tendencia a sobreestimar valores bajos de loudness, como se observa en la figura 4.5 a la izquierda. A la derecha, las estimaciones de una segunda configuración equivalente, con penalización L2 (`regression_l2`) para mitigar ese problema. Las métricas globales obtenidas fueron similares en los dos casos. La segunda configuración, con R^2 de 0,8367, un RMSE de 1,8167 y un MAE de 1,1889, se elige al aplanar las predicciones hacia los valores extremos.

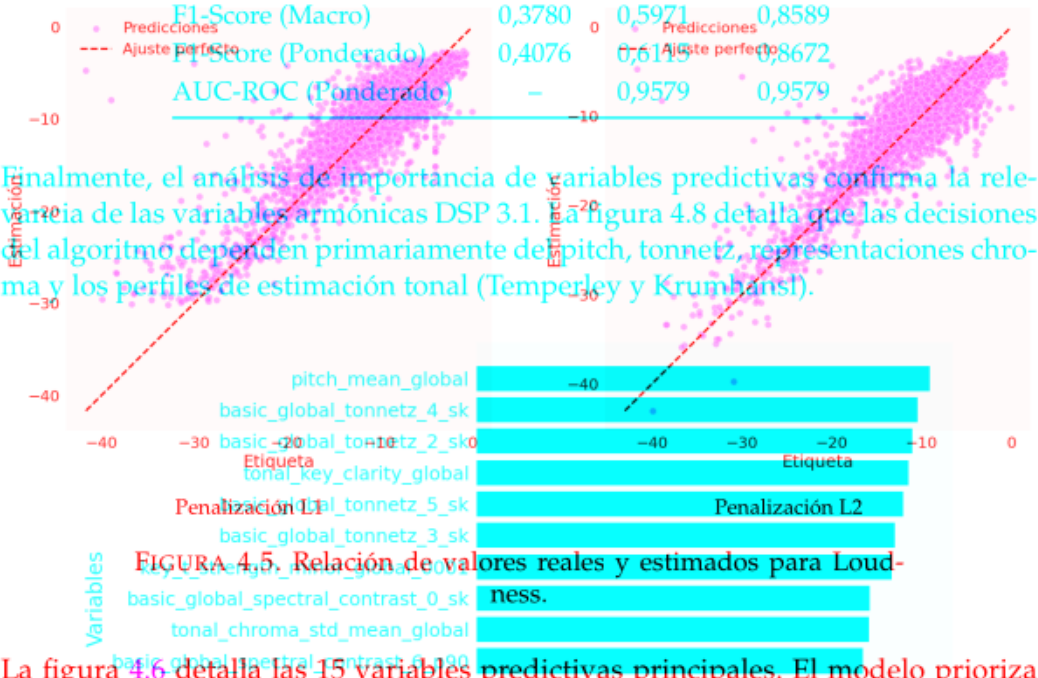


FIGURA 4.5. Relación de valores reales y estimados para Loudness.

La figura 4.6 detalla las 15 variables predictivas principales. El modelo prioriza el factor de cresta y los LUFS, seguidos de estadísticos de las representaciones DSP tradicionales como el RMS y el Centroide espectral. A diferencia de las variables perceptibles analizadas en la sección anterior, en este modelo sí destaca la presencia de un embedding de PANNs inference dentro del conjunto de mayor relevancia.

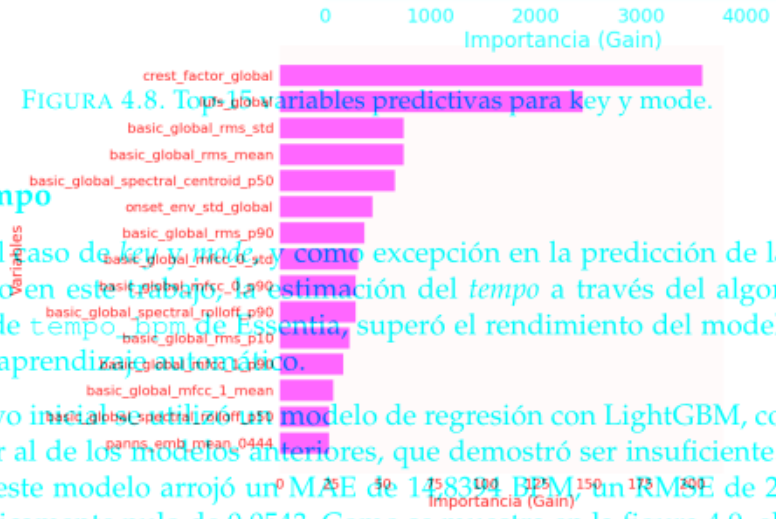


FIGURA 4.6. Top-15 variables predictivas para loudness.

4.1.4. Tempo

Contrario al caso de *key* y *mode*, y como excepción en la predicción de las variables objetivo en este trabajo, la estimación del *tempo* a través del algoritmo de extracción de `tempo_bpm` de Essentia, superó el rendimiento del modelado con técnicas de aprendizaje automático.

En un ensayo inicial utilizando un modelo de regresión con LightGBM, con un diseño similar al de los modelos anteriores, que demostró ser insuficiente. La evaluación de este modelo arrojó un MAE de 14,8334 BPM, un RMSE de 26,7187 y un R^2 prácticamente nulo de 0,0543. Como se muestra en la figura 4.9, el modelo de regresión sólo presenta capacidad predictiva alrededor de la moda (aproximadamente 120 BPM). Fuera de ese centro, el algoritmo sobrestima los valores inferiores y subestima los superiores.

4.1.3. Key y mode

Se implementó un modelo LightGBM que optimiza la pérdida logarítmica multiclase (`multi_logloss`). Durante el entrenamiento, la función de early stopping detuvo el proceso en la iteración 736 al estabilizarse la pérdida de validación en 1,1926.

La evaluación del modelo alcanzó una exactitud (accuracy) de 0,6106 y un F1-Score ponderado de 0,6115. Destaca un AUC-ROC promedio de 0,9589, lo que indica buena capacidad de discriminación entre las 24 clases. Adicionalmente, el accuracy del top-3 alcanzó 0,8671, donde la tonalidad correcta se encuentra casi siempre entre las tres primeras predicciones del sistema. La figura 4.7 presenta la matriz de confusión absoluta.

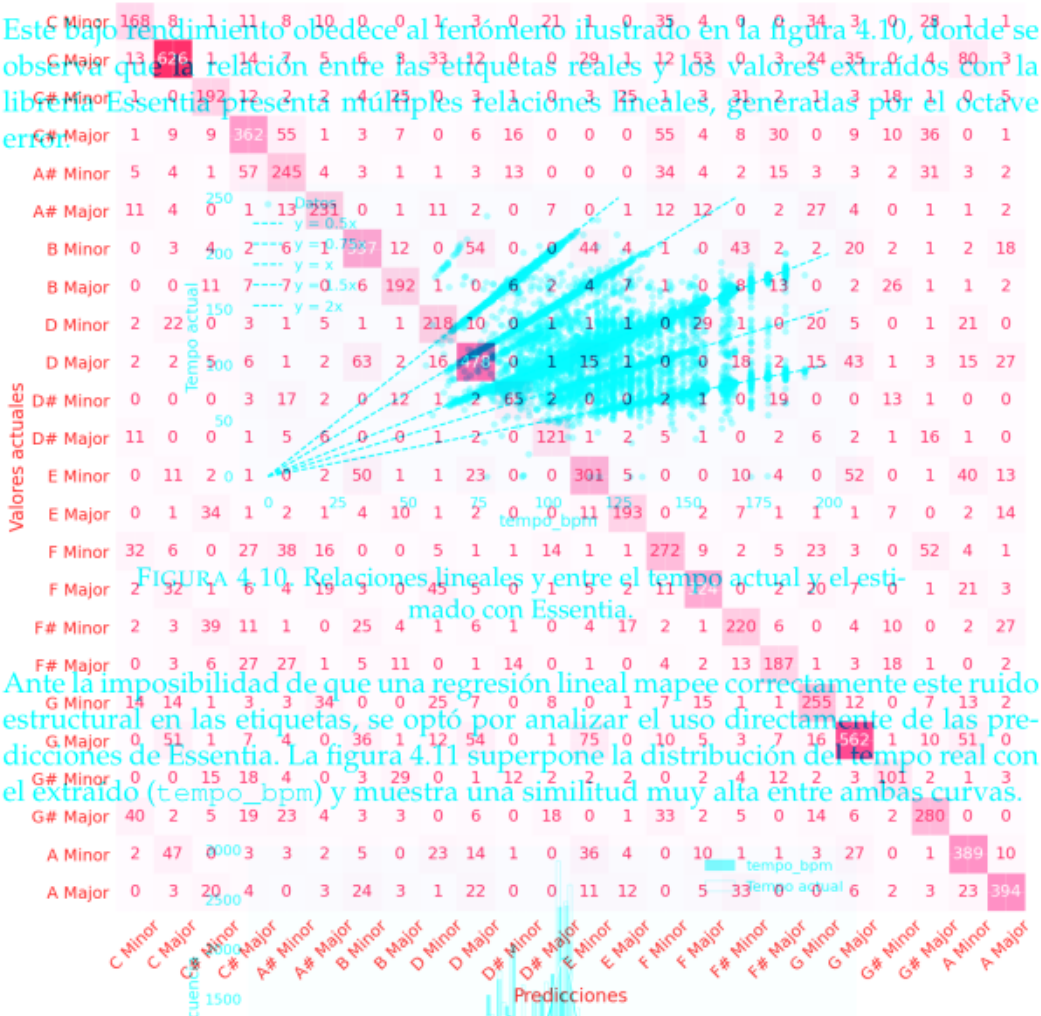


FIGURA 4.7. Matriz de confusión de key y mode.

El análisis de los errores revela una fuerte coherencia armónica: cuando el modelo se equivoca en la predicción principal con datos no vistos, las confusiones más frecuentes ocurren con la tonalidad relativa (mayor o menor correspondientes) o con las tonalidades dominante y subdominante de la escala mayor o menor correspondiente.

Para contextualizar este desempeño, la tabla 4.2 contrasta los resultados del extractor de tonalidad de la biblioteca Essentia evaluado en el dataset de Plusyc,

con las predicciones top-1 y top-3 del modelo LightGBM. El modelo supera a la herramienta de referencia en todas las métricas.

TABLA 4.2. Métricas de Essentia y del modelo de key y mode.

Métrica	Essentia	Modelo LightGBM	
		Media de error	
	Cantidad	Top-1	Top-3
Accuracy	0,4291	0,6106	0,8671
F1-Score (Macro)	0,3780	0,5971	0,8589
F1-Score (Ponderado)	0,4076	0,6115	0,8672
AUC-ROC (Ponderado)	–	0,9579	0,9579

Finalmente, el análisis de importancia de variables predictivas confirma la relevancia de las variables armónicas DSP. La figura 4.8 detalla que las decisiones del algoritmo dependen primariamente del pitch, tonnetz, representaciones chroma, espectrales y los perfiles de estimación tonal de Temperley.

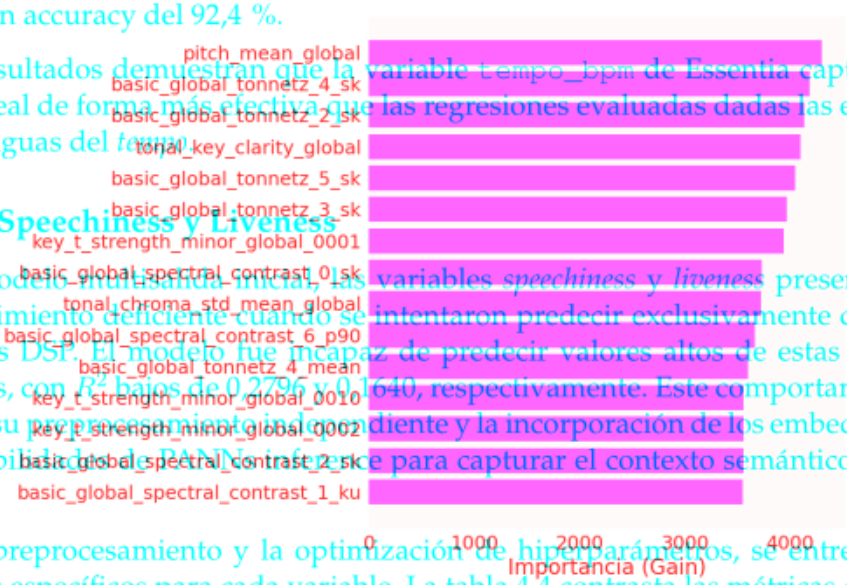
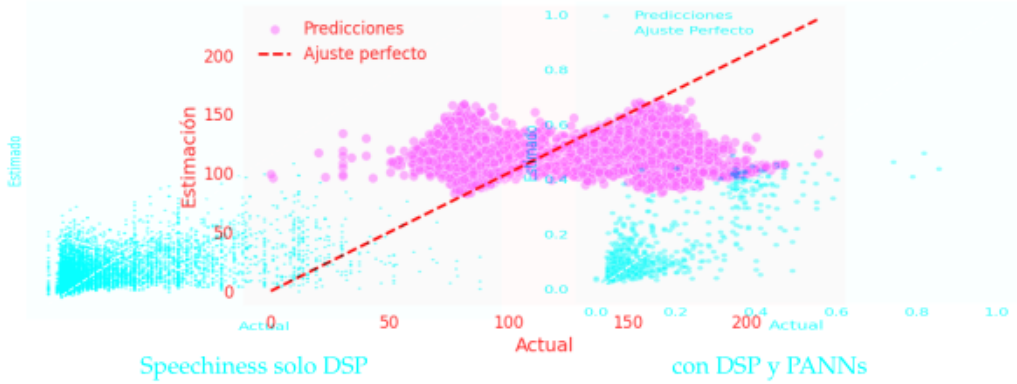


FIGURA 4.8. Top-15 variables predictivas para key y mode.

4.1.4. Tempo

Contrario al caso de *key* y *mode*, y como excepción en la predicción de las variables objetivo en este trabajo, la estimación del *tempo* a través del algoritmo de extracción de `tempo_bpm` de Essentia, superó el rendimiento del modelado con técnicas de aprendizaje automático.

En un ensayo inicial se utilizó un modelo de regresión con LightGBM, con un diseño similar al de los modelos anteriores, que demostró ser insuficiente. La evaluación de este modelo arrojó un MAE de 14,8394 BPM, un RMSE de 26,7187 y un R^2 prácticamente nulo de 0,0543. Como se muestra en la figura 4.9, el modelo de regresión solo presenta capacidad predictiva alrededor de la moda (aproximadamente 120 BPM). Fuera de ese centro, el algoritmo sobrestima los valores inferiores y subestima los superiores.



Speechiness solo DSP con DSP y PANNs

FIGURA 4.9. Relación de valores actuales y estimados para tempo.

Este bajo rendimiento obedece al fenómeno ilustrado en la figura 4.10, donde se observa que la relación entre las etiquetas reales y los valores extraídos con la biblioteca Essentia presenta múltiples relaciones lineales, generadas por el octave error.

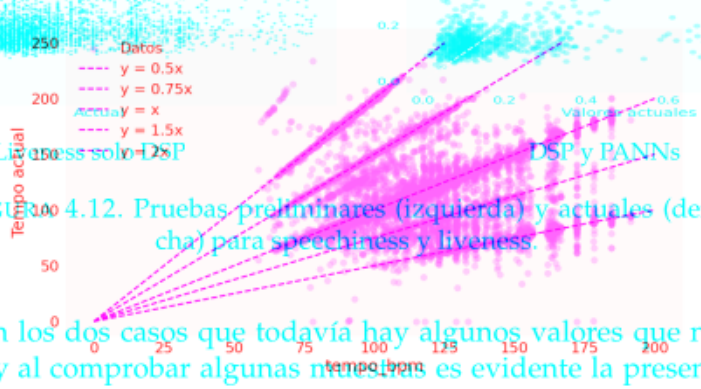


FIGURA 4.10. Relaciones lineales y entre el tempo actual y el estimado con Essentia.

Ante la imposibilidad de que una regresión lineal mapee correctamente este ruido estructural en las etiquetas, se optó por analizar el uso directamente de las predicciones de Essentia. La figura 4.11 superpone la distribución del tempo real con el extraído (`tempo_bpm`) y muestra una similitud muy alta entre ambas curvas.

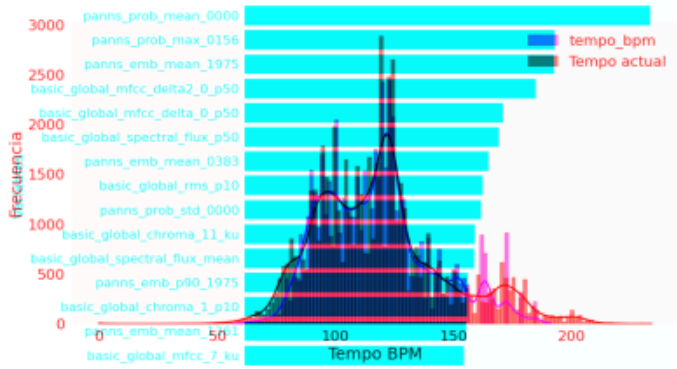


FIGURA 4.11. Histogramas de frecuencias del tempo actual y el estimado con Essentia.

FIGURA 4.13. top-15 variables predictivas para speechiness.

Se analizó la frecuencia de las proporciones métricas principales observadas y el error asociado. La tabla 4.3 desglosa esta información, donde se observa que el

85,3 % de las canciones presentan una relación directa ($y \approx x$). En este grupo mayoritario, la diferencia media es de apenas 0,0091 BPM con desviación estándar $\sigma = 2{,}17$.

TABLA 4.3. Resultados de la estimación de tempo de Essentia.

Relación (y vs. x)	Proporción (%)	Cantidad	Media de error	Desv. Est. (σ)
$y \approx x$	85,3	45.002	0,0091	2,1743
$y \approx 0,5x$	5,7	3.003	-0,0734	3,3490
$y \approx 2x$	3,2	1.682	-0,1027	2,5812
$y \approx 0,67x$	3,0	1.604	7,3190	6,9215
$y \approx 1,5x$	2,8	1.472	-3,8125	8,2052

Al evaluar el MAE en el conjunto completo de datos se obtiene un valor de 12,81 BPM y un accuracy del 80,3 %, con tolerancia de ± 5 BPM. Este error se puede explicar porque contiene el octave error. Al recalcular las métricas dando como similares las proporciones (mitades, dobles, etc.), el MAE global desciende a 1,45 BPM, con accuracy del 92,4 %.

Estos resultados demuestran que la variable `tempo_bpm` de Essentia captura el tempo real de forma más efectiva que las regresiones evaluadas dadas las etiquetas ambiguas del *tempo*.

4.1.5. Speechiness y liveness

En el modelo multisalida inicial, las variables *speechiness* y *liveness* presentaron un rendimiento deficiente cuando se intentaron predecir exclusivamente con las variables DSP. El modelo fue incapaz de predecir valores altos de estas distribuciones, con R^2 bajos de 0,2796 y 0,1640, respectivamente. Este comportamiento motivó su preprocesamiento independiente y la incorporación de los embeddings y probabilidades de PANNs inference para capturar el contexto semántico necesario.

Tras el preprocesamiento y la optimización de hiperparámetros, se entrenaron modelos específicos para cada variable. La tabla 4.4 contrasta las métricas del experimento preliminar frente a los modelos actuales, con mejora en el rendimiento. En *speechiness*, el R^2 subió a 0,6748. Por su parte, el modelo de *liveness* alcanzó un R^2 de 0,6318.

TABLA 4.4. Métricas en el modelado de speechiness y liveness.

Variable	R^2 (↑)		MAE (↓)		RMSE (↓)	
	Inicial	Definitivo	Inicial	Definitivo	Inicial	Definitivo
Speechiness	0,2796	0,6748	0,0570	0,0405	0,1024	0,0675
Liveness	0,1640	0,6318	0,0978	0,0811	0,1392	0,1301

La mejora se observa gráficamente en la figura 4.12. A diferencia del modelo inicial, las estimaciones del modelo actual logra predicciones acorde a las definiciones. En *speechiness* a la izquierda, el modelo demuestra capacidad para detectar audios con alto contenido de voz hablada, como rap o intermedios vocales. En *liveness* a la derecha, logra reconocer la presencia de público en vivo.

Este patrón evidencia inicio de sobreajuste, esperado dada la alta densidad de clases acústicamente similares, de ahí la importancia del early stopping.

La figura 4.16 presenta la matriz de confusión normalizada. Debido a la densidad de las 197 clases, la diagonal principal resalta la concentración de aciertos, y también muestra un bajo nivel de confusión afuera de la diagonal.

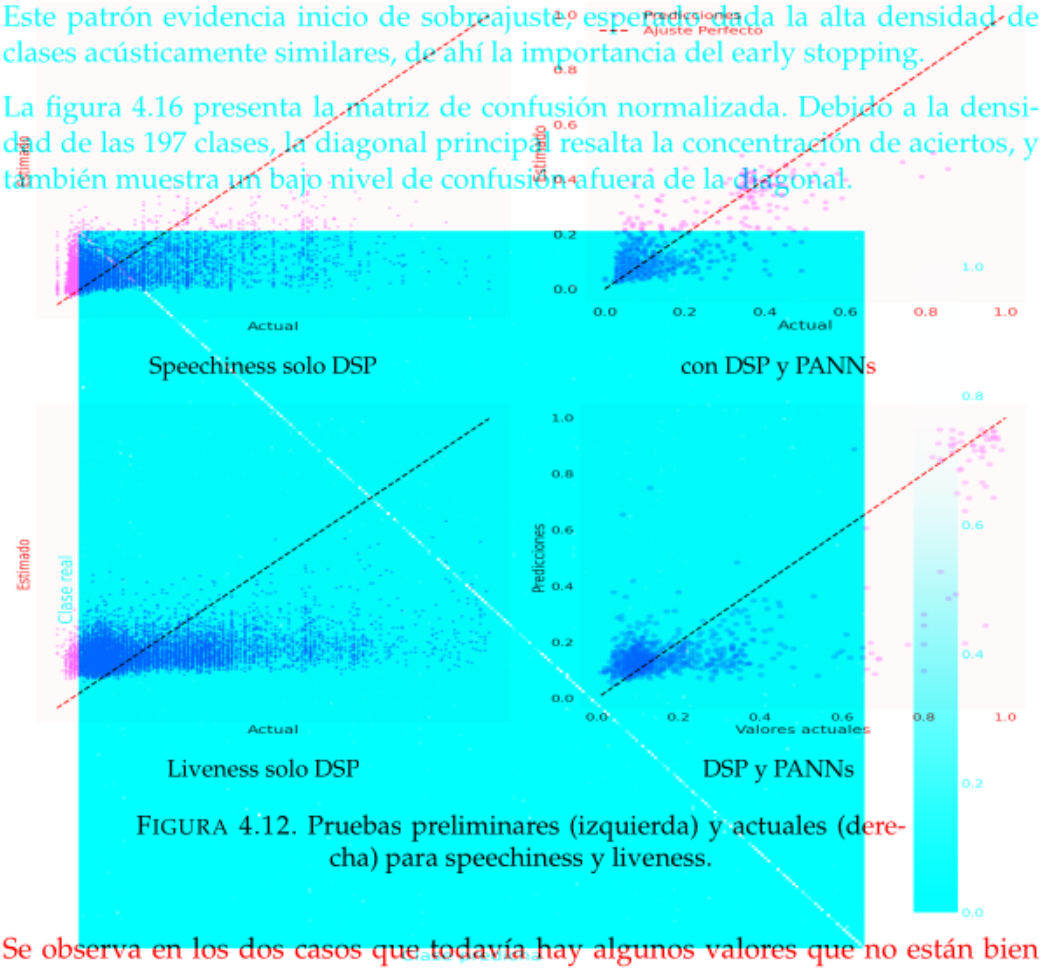


FIGURA 4.12. Pruebas preliminares (izquierda) y actuales (derecha) para speechiness y liveness.

Se observa en los dos casos que todavía hay algunos valores que no están bien clasificados, y al comprobar algunas muestras es evidente la presencia de ruido en los valores, esto se deja para un trabajo futuro sobre la limpieza de datos con apoyo en PANNs inference. El análisis de la importancia de características confirma el impacto del contexto semántico introducido por PANNs.

FIGURA 4.16. Matriz de confusión normalizada para la clasificación de 197 géneros musicales.

En la figura 4.17, se observa el top-15 de las características predictivas.

Para *speechiness*, en la figura 4.13 se observa que las variables de mayor peso son las probabilidades de las clases 0000 (*speech*) y 0156 (*song*), se prueba que el modelo las tiene en cuenta para diferenciar entre la voz hablada y el canto melódico.

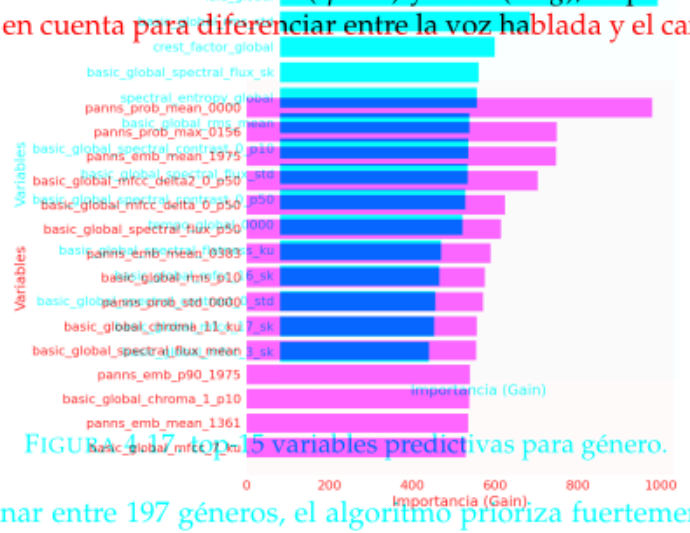


FIGURA 4.17. Top-15 variables predictivas para género.

Para discriminar entre 197 géneros, el algoritmo prioriza fuertemente la intensidad (LUFS y RMS) y las características macro del espectro.

FIGURA 4.13. Top-15 variables predictivas para speechiness.

Por otro lado, en la figura 4.14, las variables con mayor importancia para la predicción de *liveness*, están relacionadas con las probabilidades de público en vivo y se observa en las clases 0022 (*applause*), 0021 (*cheering*) y 0024 (*crowd*).

4.2. Ejemplo de inferencia

Pendiente

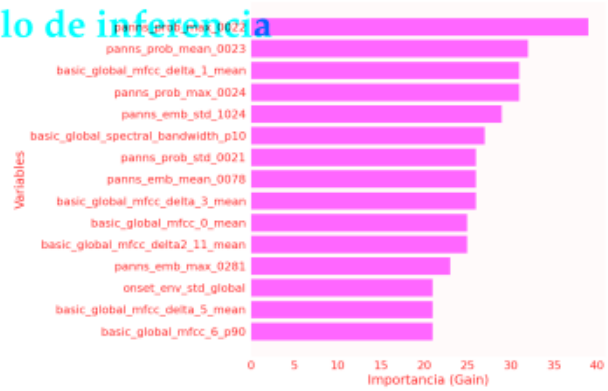


FIGURA 4.14. Top-15 variables predictivas para liveness.

4.1.6. Suggested genres

Para el problema de clasificar 197 géneros y su evaluación, se priorizó la capacidad del modelo para generar sugerencias coherentes. Las métricas globales sobre el conjunto de prueba alcanzaron un accuracy y un F1-Score cercanos a 0,47. Si bien predecir el género exacto (top-1) arroja un 46,74 % de accuracy, el rendimiento del algoritmo escala muy bien al analizar estimaciones más probables. El accuracy top-3 alcanza un 68,32 %, y el accuracy top-5, el 77,31 %. Esto significa que, en casi 8 de cada 10 casos, el modelo ubica el género correcto del audio entre sus primeras cinco sugerencias.

Para el aprendizaje del espacio de clasificación tan complejo, se analizó la evolución de la pérdida logarítmica (`multi_logloss`). La figura 4.15 ilustra las curvas, con descenso continuo en el error de entrenamiento hasta valores cercanos a cero. La curva de validación encuentra su punto de mínima pérdida alrededor de la iteración 150, con un valor cercano a 2,10, y posteriormente se estanca.

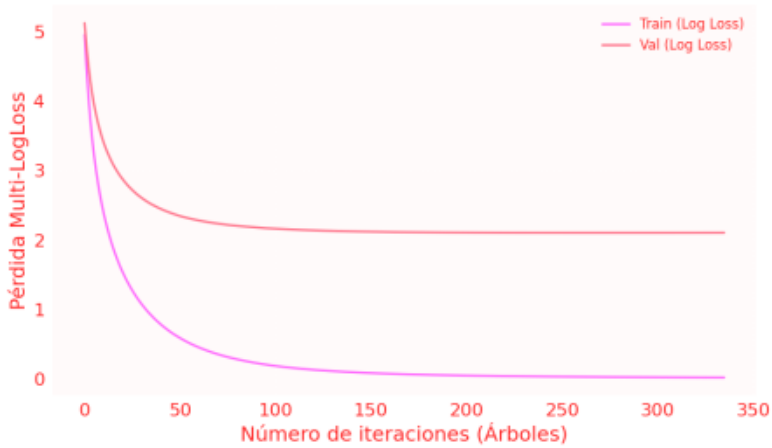


FIGURA 4.15. Curvas de aprendizaje del modelo para la clasificación de géneros.

Este patrón evidencia inicio de sobreajuste, esperado dada la alta densidad de clases acústicamente similares, de ahí la importancia del early stopping.

La figura 4.16 presenta la matriz de confusión normalizada. A pesar de la densidad de las 197 clases, la diagonal principal resalta la concentración de aciertos, y también muestra un bajo nivel de confusión afuera de la diagonal.

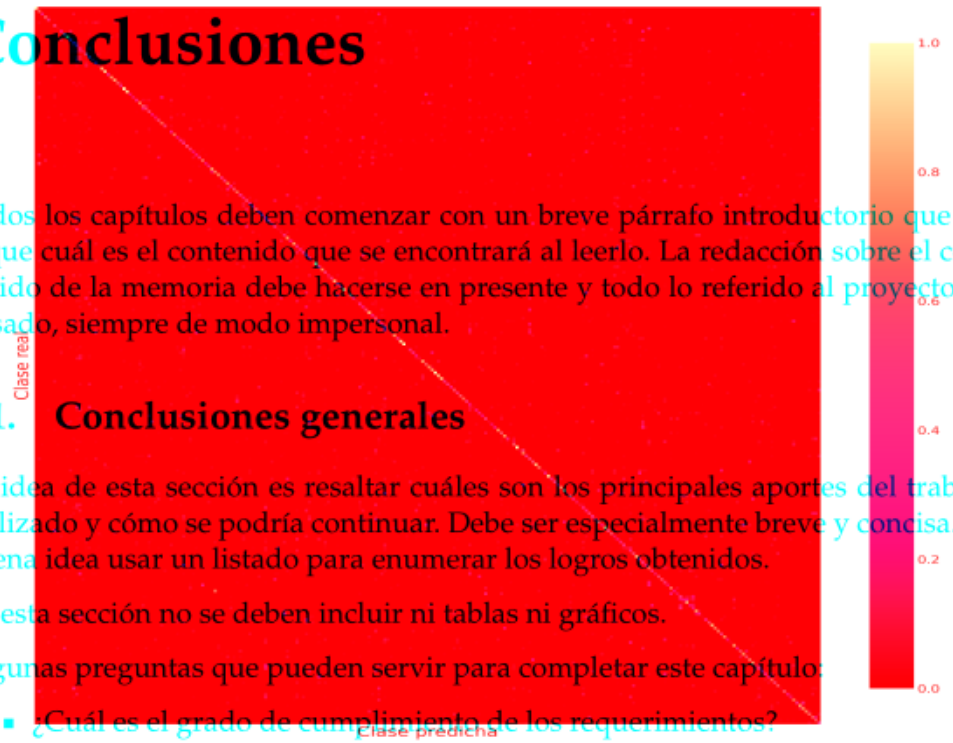


FIGURA 4.16. Matriz de confusión normalizada para la clasificación de 197 géneros musicales.

En la figura 4.17, se observa el top-15 de las características predictivas.

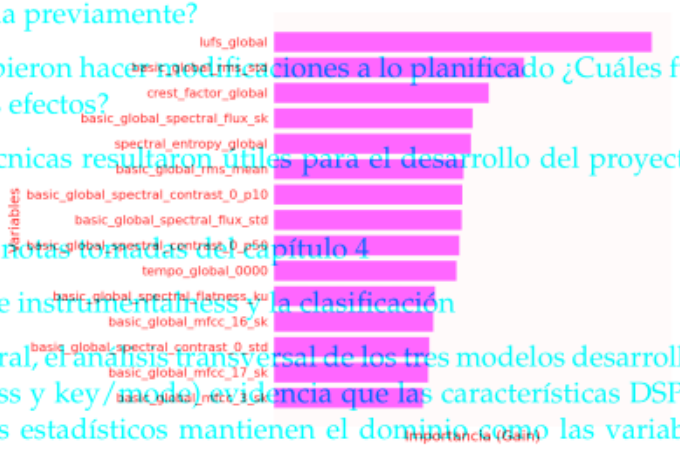


FIGURA 4.17. Top-15 variables predictivas para género.

Para discriminar entre 197 géneros, el algoritmo prioriza fuertemente la intensidad (LUFS y RMS) y las características espectrales.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

WIP Algunas notas tomadas del capítulo 4

- Acordarse de instrumentalness y la clasificación

- A nivel general, el análisis transversal de los tres modelos desarrollados (perceptibles, loudness y key/modo) evidencia que las características DSP tradicionales y sus diversos estadísticos mantienen el dominio como las variables de mayor impacto predictivo, complementadas transversalmente por los embeddings semánticos (PANNs) que operan en conjunto para refinar la generalización.

- Se observa que los modelos tienden a sobreajustarse a partir de la época 10 aproximadamente. Por otro lado, en los espectrogramas es evidente la presencia de ruido en los valores, esto se deja para un trabajo futuro sobre la limpieza de datos con apoyo en PANNs inference

4.1.7. Primary y secondary moods

El diseño de estas categorías fue similar al caso de géneros, aunque presenta un grado de subjetividad aún mayor reflejado en los valores actuales, lo que impacta las métricas obtenidas.

Para *primary mood*, el modelo alcanzó un accuracy global de 0,2488 y un F1-Score macro de 0,2416. Al evaluar rangos más amplios: el accuracy escala al 55,81 % en el top-3 y al 73,64 % en el top-5. La figura 4.18 presenta la matriz de confusión.

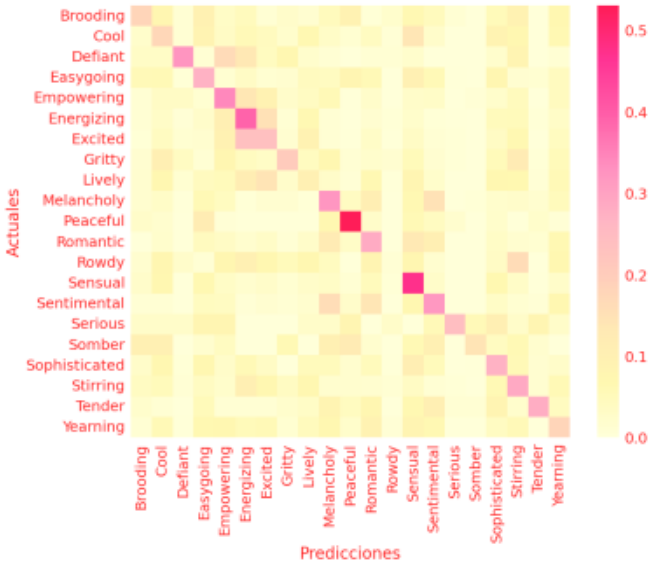


FIGURA 4.18. Matriz de confusión para primary mood.

Se distingue la diagonal principal y también las clases más difíciles de predecir, como *somber*, que se confinde con *melancholy* y *sentimental*; y *rowdy*, con *excited*, *empowering* y *stirring*; *excited* con *energizing* y *lively* con *excited*. Estos patrones reflejan la subjetividad inherente en las etiquetas, más allá del desbalance de clases. Si bien estas métricas (top-1) son modestas, el modelo demuestra capacidad de aprendizaje. Para complementar el análisis, la figura 4.19 presenta las curvas ROC bajo el esquema uno contra todos (One-vs-Rest).

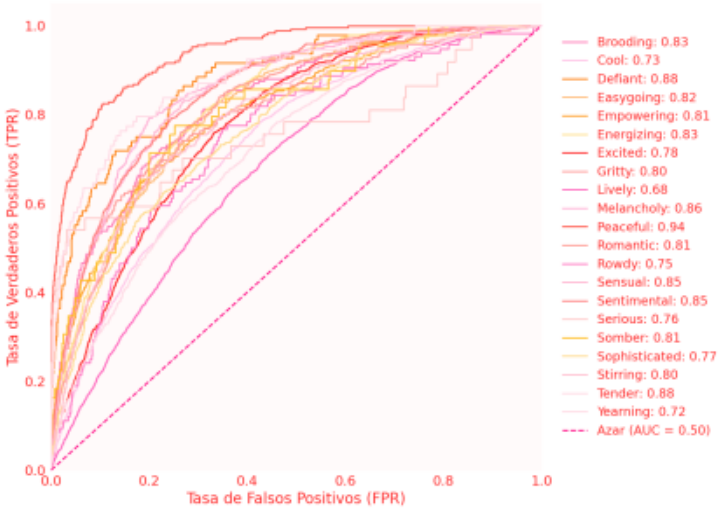


FIGURA 4.19. Curvas ROC y valores AUC para primary mood.

Los valores del área bajo la curva (AUC) demuestran capacidad de discriminación en la mayoría de los casos. Destacan clases como *peaceful* (0,94) y *tender* (0,88) con alta separabilidad, frente a *lively* (0,68). El algoritmo logra modelar distribuciones de probabilidad a pesar de la ambigüedad de las etiquetas.

El modelo de *secondary mood*, con 102 clases, evidenció un comportamiento similar. El accuracy directo fue del 18,75 % con un F1-Score macro de 0,1458. No obstante, al evaluar las sugerencias, el accuracy subió progresivamente al 46,57 % en el top-5 y a 79,00 % recién en el top-20. la figura 4.20 ilustra la matriz de confusión normalizada de *secondary mood*, donde también se muestra una diagonal principal que indica aprendizaje. Y algunos indicios de lineas verticales muestran tendencia a preferir las clases predominantes.

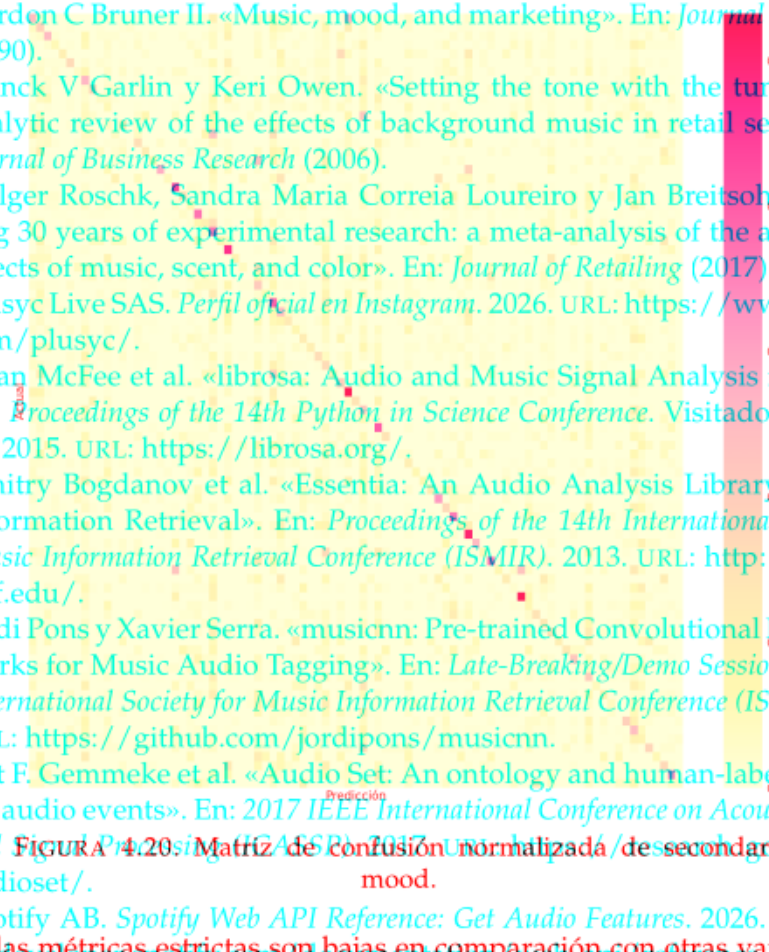


FIGURA 4.20. Matriz de confusión normalizada de secondary mood.

Aunque las métricas estrictas son bajas en comparación con otras variables analizadas, la evolución positiva en las precisiones top-K confirma que ambos modelos lograron extraer señal de las variables de entrada y que las predicciones se agrupan consistentemente dentro de espectros de estados de ánimo similares. Este comportamiento resulta útil en el contexto de Plusyc.

Secondary mood está presente como característica pero actualmente no se usa como un filtro para ambientación en Plusyc. Esto también explica el desbalance, los curadores tienden a favorecer algunos estados de ánimo secundarios más que otros.

Bibliografía

[1] J. Stephen Downie. «Music Information Retrieval». En: *Annual Review of Information Science and Technology* (2003).

[2] Michael A. Casey et al. «Content-Based Music Information Retrieval: Current Directions and Future Challenges». En: *Proceedings of the IEEE* (2008).

[3] Gordon C Bruner II. «Music, mood, and marketing». En: *Journal of marketing* (1990).

[4] Franck V Garlin y Keri Owen. «Setting the tone with the tune: A meta-analytic review of the effects of background music in retail settings». En: *Journal of Business Research* (2006).

[5] Holger Roschk, Sandra Maria Correia Loureiro y Jan Breitsohl. «Calibrating 30 years of experimental research: a meta-analysis of the atmospheric effects of music, scent, and color». En: *Journal of Retailing* (2017).

[6] Plusyc Live SAS. *Perfil oficial en Instagram*. 2026. URL: https://www.instagram.com/plusyc/.

[7] Brian McFee et al. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference*. Visitado el 2026-03-20. 2015. URL: https://librosa.org/.

[8] Dmitry Bogdanov et al. «Essentia: An Audio Analysis Library for Music Information Retrieval». En: *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*. 2013. URL: http://essentia.upf.edu/.

[9] Jordi Pons y Xavier Serra. «musicnn: Pre-trained Convolutional Neural Networks for Music Audio Tagging». En: *Late-Breaking/Demo Session of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. 2019. URL: https://github.com/jordipons/musicnn.

[10] Jort F. Gemmeke et al. «Audio Set: An ontology and human-labeled dataset for audio events». En: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2017. URL: https://research.google.com/audioset/.

[11] Spotify AB. *Spotify Web API Reference: Get Audio Features*. 2026. URL: https://developer.spotify.com/documentation/web-api/reference/get-audio-features.

[12] Gracenote Inc. *Gracenote Music Data: Sonic Attributes and Mood*. 2026. URL: https://gracenote.com/products/audio-data/.

[13] Tunebat. *Tunebat: Song Key, BPM and Audio Features Database*. 2026. URL: https://tunebat.com/.

[14] George Tzanetakis y Perry Cook. «Musical genre classification of audio signals». En: *IEEE Transactions on Speech and Audio Processing* (2002).

[15] Michaël Defferrard et al. «FMA: A Dataset for Music Analysis». En: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017.

[16] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

4.2. Ejemplo de inferencia

Para demostrar el funcionamiento integral del sistema, se ejecutó una inferencia cuya entrada es la canción de Radiohead ilustrada en los primeros capítulos y que no está en el catálogo de Plusyc. El archivo de audio se procesó a través de de la API. A continuación, se presenta la respuesta en formato JSON:

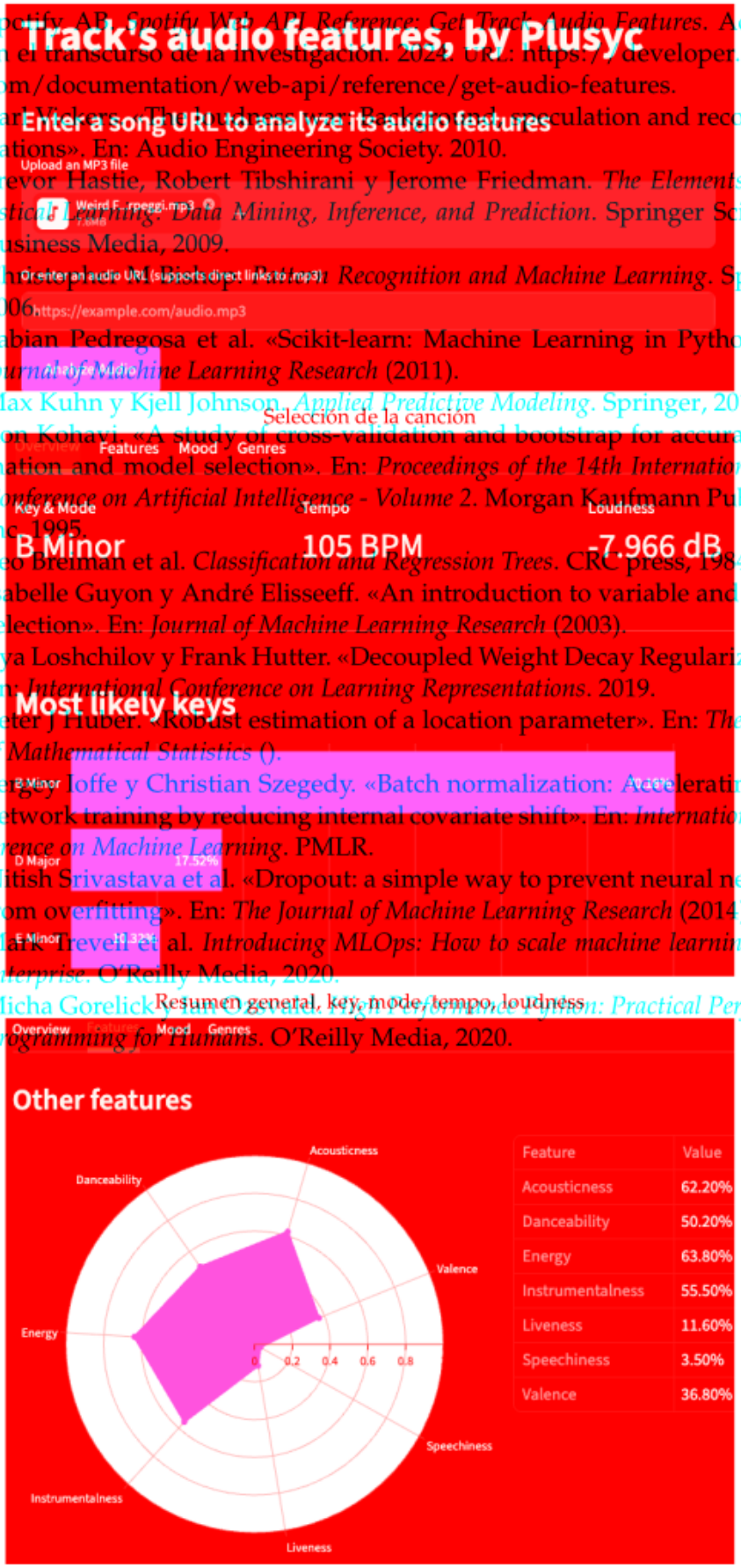
```
{
  "success": true,
  "data": {
    "Key": {
      "value": [
        { "key": 11, "mode": 0, "key_mode": "B Minor", "prob": "70.18%" },
        { "key": 2, "mode": 1, "key_mode": "D Major", "prob": "17.52%" },
        { "key": 4, "mode": 0, "key_mode": "E Minor", "prob": "10.32%" }
      ],
    "Tempo": {
      "value": 105
    },
    "Loudness": {
      "value": -7.966
    },
    "Fivep": {
      "value": {
        "acousticness": 0.622, "danceability": 0.502, "energy":
0.638, "instrumentalness": 0.555, "valence": 0.368
    },
    "Liveness": {
      "value": 0.116
    },
    "Speechiness": {
      "value": 0.035
    },
    "MoodPrimary": {
      "top_5":
        "Yearning 18.20%, Stirring 13.95%, Melancholy 9.87%, Brooding:
8.87%, Rowdy: 7.67%"
    },
    "MoodSecondary": {
      "top_5":
        "Cool Melancholy 6.30%, Lyrical Sentimental 4.80%, Awakening
Stately 4.64%, Light Melancholy 4.55%, Gentle Bittersweet 4.17%"
    },
    "GenreTarget": {
      "top_5":
        "Mainstream 90s and 00s Rock, Alternative Rock, General Latin
Rock, General Downtempo, General Indie Rock"
    }
  }
}
```

CÓDIGO 4.1. Respuesta JSON de inferencia

Las figuras 4.21 y 4.22 exponen una representación visual de estos resultados en una interfaz de usuario.

Si se comparan los resultados de la inferencia con los resultados de Tunebat de la figura 1.1, se puede observar alta similitud en los datos.

[37] Spotify AB. *Spotify Web API Reference: Get Track Audio Features*. Accedido en el transcurso de la investigación. 2024. URL: https://developer.spotify.com/documentation/web-api/reference/get-audio-features.
[38] Earl Vickers. «The Loudness War: Background, Speculation and recommendations». En: Audio Engineering Society. 2010.
[39] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, 2009.
[40] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
[41] Fabian Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* (2011).
[42] Max Kuhn y Kjell Johnson. *Applied Predictive Modeling*. Springer, 2013.
[43] Ron Kohavi. «A study of cross-validation and bootstrap for accuracy estimation and model selection». En: *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*. Morgan Kaufmann Publishers Inc, 1995.
[44] Leo Breiman et al. *Classification and Regression Trees*. CRC press, 1984.
[45] Isabelle Guyon y André Elisseeff. «An introduction to variable and feature selection». En: *Journal of Machine Learning Research* (2003).
[46] Ilya Loshchilov y Frank Hutter. «Decoupled Weight Decay Regularization». En: *International Conference on Learning Representations*. 2019.
[47] Peter J Huber. «Robust estimation of a location parameter». En: *The Annals of Mathematical Statistics* ().
[48] Sergey Ioffe y Christian Szegedy. «Batch normalization: Accelerating deep network training by reducing internal covariate shift». En: *International Conference on Machine Learning*. PMLR.
[49] Nitish Srivastava et al. «Dropout: a simple way to prevent neural networks from overfitting». En: *The Journal of Machine Learning Research* (2014).
[50] Mark Treveil et al. *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media, 2020.
[51] Micha Gorelick y Ian Ozsvald. *High Performance Python: Practical Performant Programming for Humans*. O'Reilly Media, 2020.

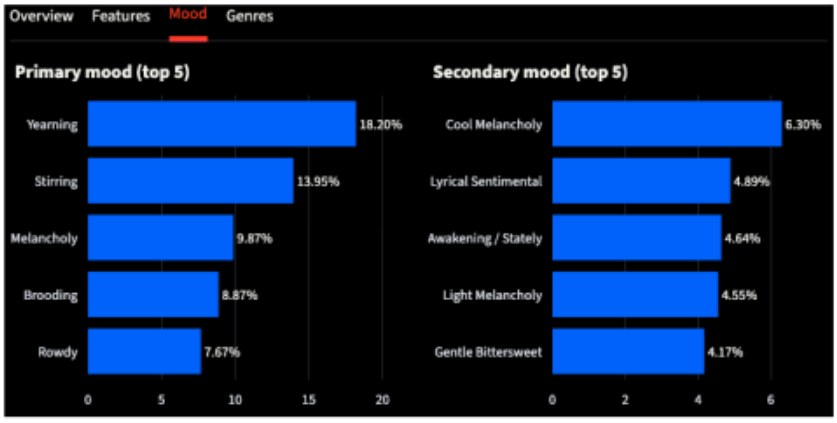


Características perceptibles

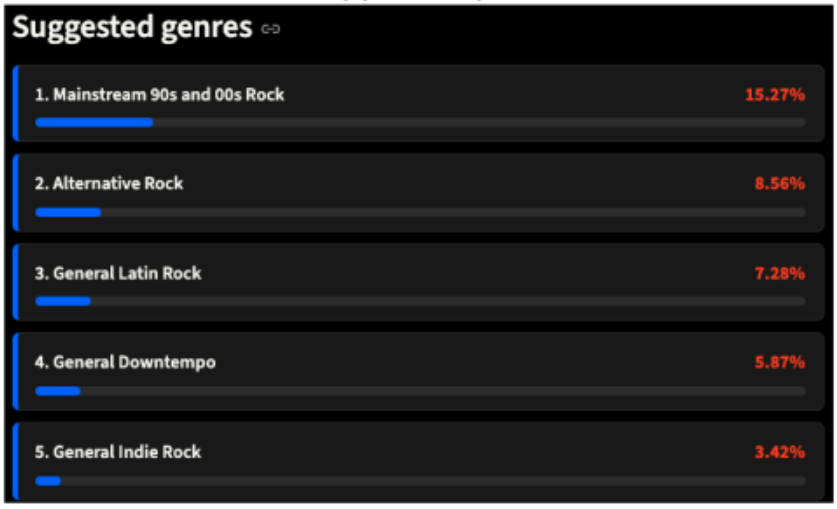
FIGURA 4.21. Visualización de los resultados de inferencia en una interfaz web (parte 1).

El único valor que está desfasado en este caso es el *tempo*, para el que no se usó un modelo de inferencia desarrollado como parte de este trabajo, sino que curiosamente se usó el algoritmo de extracción de Essentia. Además, el desfase ocurre en proporción $y \approx 0,67x$ en relación al valor real, posibilidad que se sugirió en la sección 4.1.4.

Los estados de ánimo y géneros también muestran resultados coherentes con los esperados.



Primary y secondary moods



Suggested genres

FIGURA 4.22. Visualización de los resultados de inferencia en una interfaz web (parte 2).

Capítulo 5

Conclusiones

Este capítulo presenta un resumen de los logros y las dificultades para llegar al estado actual de la solución y sugiere una serie de siguientes pasos para la evolución del sistema.

5.1. Conclusiones generales

En este trabajo se propuso diseñar e implementar un sistema capaz de predecir características musicales a partir de audio de canciones. El objetivo principal fue automatizar un proceso tradicionalmente manual y laborioso de clasificación musical y reducir la dependencia de servicios de tercecros para evitar costos recurrentes. El trabajo está en producción, clasifica las canciones que ingresan al sistema, es funcional en el contexto de una empresa en crecimiento.

En relación a los objetivos específicos, se puede concluir que:

- Se cumplieron los requerimientos técnicos al lograr la caracterización de las variables solicitadas a partir de los datos de Plusyc.
- La extracción de variables acústicas DSP sirvió para las predicciones, y la inclusión de embeddings y probabilidades de PANNs fue lo que permitió resolver las variables semánticas complejas. Sin esto último, no se habría completado el trabajo.
- Se aplicaron acciones para mitigar problemas en los datos. Fue necesario aplicar técnicas de preprocesamiento; se cortó el histórico en una fecha específica para aislar un cambio en la distribución de los datos.
- Se entrenaron modelos de aprendizaje automático para predecir las características musicales y la evaluación de las métricas de rendimiento dan resultados productivos, acorde al análisis y diseño realizados.
- La decisión de usar bibliotecas como Librosa y PANNs inference y modelos como redes MLP y ensambles de árboles, permitió no depender de licencias ni pagar costos por consultas a modelos de terceros, también facilitó el trabajo al no requerir de arquitecturas complejas.
- En específico, la clasificación de estados de ánimo y géneros sugiere que la ambigüedad humana al etiquetar afecta el aprendizaje. Estos resultados indican que el equipo de curaduría necesita estandarizar los criterios para la ingesta manual de datos.
- Para la detección de público en vivo y habla en canciones, se requirió de limpieza de ruido por falsas etiquetas apoyada en un modelo preentrenado

con el conjunto de datos AudioSet, creado para la generación automática de subtítulos como [Aplausos] o [Rapping].

- Completar el trabajo tomó más tiempo del planificado en el cronograma original. Hubo una subestimación del esfuerzo, dado que la cantidad de modelos y el proceso de aprendizaje requirieron más iteraciones de las previstas.

5.2. Próximos pasos

Los diseños actuales quedan disponibles y se pueden usar para iteraciones futuras. Algunos aspectos que se podrían mejorar o explorar son:

- El uso de las inferencias de PANNs y la exploración de otros modelos preentrenados, para mejorar la limpieza de algunos datos y corregir audios mal etiquetados. Como ejemplos, el *speechiness*, *liveness* y la predicción de *instrumentalness*: el modelo actual logra diferenciar una canción instrumental de una que no lo es, pero los valores en el medio no son claros. Se sugiere revisar puntualmente, ya que presenta ruido en los valores intermedios por falta de definiciones claras.
- También se sugiere iterar sobre el *tempo* para buscar una solución más robusta que el algoritmo de DSP.
- Se propone empezar a usar el estado de ánimo secundario como filtro en Plusyc para mejorar la precisión de las recomendaciones musicales.
- El sistema se puede ajustar para el despliegue bajo el modelo de software como servicio y así permitir ofrecer el servicio al público interesado.
- El equipo de curadores sigue clasificando música. Se contempla automatizar un reentrenamiento periódico para que el sistema aprenda nuevos géneros cada vez que una clase alcance un mínimo de 100 muestras, con el uso de tecnologías como Airflow y MLflow para optimizar estos procesos, hoy manuales.
- Se podría evaluar a futuro el uso de arquitecturas basadas en transformadores, si se cuenta con mayor capacidad de cómputo, para mejorar la clasificación de clases densas. También, complementar el sistema con modelos de aprendizaje profundo enfocados exclusivamente en música.

Bibliografía

[1] J. Stephen Downie. «Music Information Retrieval». En: *Annual Review of Information Science and Technology* (2003).

[2] Michael A. Casey et al. «Content-Based Music Information Retrieval: Current Directions and Future Challenges». En: *Proceedings of the IEEE* (2008).

[3] Gordon C Bruner II. «Music, mood, and marketing». En: *Journal of marketing* (1990).

[4] Franck V Garlin y Keri Owen. «Setting the tone with the tune: A meta-analytic review of the effects of background music in retail settings». En: *Journal of Business Research* (2006).

[5] Holger Roschk, Sandra Maria Correia Loureiro y Jan Breitsohl. «Calibrating 30 years of experimental research: a meta-analysis of the atmospheric effects of music, scent, and color». En: *Journal of Retailing* (2017).

[6] Plusyc Live SAS. *Sitio web oficial*. 2026. URL: https://www.plusyc.com.

[7] Brian McFee et al. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference*. Visitado el 2026-03-20. 2015. URL: https://librosa.org/.

[8] Dmitry Bogdanov et al. «Essentia: An Audio Analysis Library for Music Information Retrieval». En: *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*. 2013. URL: http://essentia.upf.edu/.

[9] Jordi Pons y Xavier Serra. «musicnn: Pre-trained Convolutional Neural Networks for Music Audio Tagging». En: *Late-Breaking/Demo Session of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. 2019. URL: https://github.com/jordipons/musicnn.

[10] Jort F. Gemmeke et al. «Audio Set: An ontology and human-labeled dataset for audio events». En: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2017. URL: https://research.google.com/audioset/.

[11] Spotify AB. *Spotify Web API Reference: Get Audio Features*. 2026. URL: https://developer.spotify.com/documentation/web-api/reference/get-audio-features.

[12] Gracenote Inc. *Gracenote Music Data: Sonic Attributes and Mood*. 2026. URL: https://gracenote.com/products/audio-data/.

[13] Tunebat. *Tunebat: Song Key, BPM and Audio Features Database*. 2026. URL: https://tunebat.com/.

[14] George Tzanetakis y Perry Cook. «Musical genre classification of audio signals». En: *IEEE Transactions on Speech and Audio Processing* (2002).

[15] Michaël Defferrard et al. «FMA: A Dataset for Music Analysis». En: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017.

[16] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

[17] Alexander Lerch. *An introduction to audio content analysis: Applications in signal processing and music informatics*. John Wiley & Sons, 2012.

[18] Emilia Gómez. «Tonal description of polyphonic audio for music content processing». En: *INFORMS Journal on Computing* (2006).

[19] Christopher Harte, Mark Sandler y Martin Gasser. «Detecting harmonic change in musical audio». En: *Proceedings of the 1st ACM workshop on Audio and music computing multimedia*. 2006.

[20] Qiuqiang Kong et al. «PANNs: Large-scale pretrained audio neural networks for audio pattern recognition». En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2020).

[21] Karen Simonyan y Andrew Zisserman. «Very deep convolutional networks for large-scale image recognition». En: *International Conference on Learning Representations*. 2015.

[22] Guolin Ke et al. «Lightgbm: A highly efficient gradient boosting decision tree». En: *Advances in neural information processing systems*. Vol. 30. 2017.

[23] Takuya Akiba et al. «Optuna: A next-generation hyperparameter optimization framework». En: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019.

[24] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep learning*. MIT press, 2016.

[25] Sebastián Ramírez. *FastAPI*. 2018. URL: https://github.com/fastapi/fastapi.

[26] Python Software Foundation. *Python Language Reference*. 2026. URL: https://www.python.org.

[27] Dirk Merkel. «Docker: lightweight linux containers for consistent development and deployment». En: *Linux journal* (2014).

[28] Oracle Corporation. *Oracle Cloud Infrastructure*. 2026. URL: https://www.oracle.com/cloud/.

[29] Thierry Bertin-Mahieux et al. «The million song dataset». En: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011)*. 2011.

[30] Yann LeCun et al. «Gradient-based learning applied to document recognition». En: *Proceedings of the IEEE* (1998).

[31] Bob L. Sturm. «The state of the art ten years after a state of the art: Future research in music information retrieval». En: *Journal of New Music Research* (2014).

[32] Geoffroy Peeters. *A large set of audio features for sound description (similarity and classification) in the CUIDADO project*. Inf. téc. IRCAM (Institut de Recherche et Coordination Acoustique/Musique), 2004.

[33] Andrada Olteanu. *GTZAN Dataset - Music Genre Classification*. 2020. URL: https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification.

[34] Shawn Hershey et al. «CNN architectures for large-scale audio classification». En: IEEE. 2017.

[35] ITU-R. «Algorithms to measure audio programme loudness and true-peak audio level». En: *Recommendation ITU-R BS.1770-4* (2015).

[36] Alain De Cheveigné e Hideki Kawahara. «YIN, a fundamental frequency estimator for speech and music». En: *The Journal of the Acoustical Society of America* (2002).

[37] Earl Vickers. «The loudness war: Background, speculation and recommendations». En: Audio Engineering Society. 2010.

[38] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, 2009.

[39] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[40] Fabian Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* (2011).

[41] Max Kuhn y Kjell Johnson. *Applied Predictive Modeling*. Springer, 2013.

[42] Ron Kohavi. «A study of cross-validation and bootstrap for accuracy estimation and model selection». En: *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*. Morgan Kaufmann Publishers Inc. 1995.

[43] Leo Breiman et al. *Classification and Regression Trees*. CRC press, 1984.

[44] Isabelle Guyon y André Elisseeff. «An introduction to variable and feature selection». En: *Journal of Machine Learning Research* (2003).

[45] Ilya Loshchilov y Frank Hutter. «Decoupled Weight Decay Regularization». En: *International Conference on Learning Representations*. 2019.

[46] Peter J Huber. «Robust estimation of a location parameter». En: *The Annals of Mathematical Statistics* (1964).

[47] Sergey Ioffe y Christian Szegedy. «Batch normalization: Accelerating deep network training by reducing internal covariate shift». En: *International Conference on Machine Learning*. PMLR. 2015.

[48] Nitish Srivastava et al. «Dropout: a simple way to prevent neural networks from overfitting». En: *The Journal of Machine Learning Research* (2014).

[49] Mark Treveil et al. *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media, 2020.

[50] Micha Gorelick y Ian Ozsvald. *High Performance Python: Practical Performant Programming for Humans*. O'Reilly Media, 2020.